Bachelor Thesis

A Practical Evaluation of Oversmoothing and Oversquashing in Graph Neural Networks

Malaz Natouf

Learning on Graphs
RWTH Aachen University
Aachen, Germany
malaz.natouf@rwth-aachen.de

Professors

Prof. Dr. Christopher Morris
Prof. Dr. Michael Schaub

Supervisor

Timo Stoll

Registration Date

28 August 2026

Submission Date

Abstract:

Contents

1 Introduction **4**

1.1 Motivation 4

1.2 Research Questions 4

2 Background **4**

2.1 Message Passing and Receptive Fields 4

2.2 Depth versus Hidden Dimension 5

2.3 Homophily and Heterophily 7

2.4 Oversmoothing 9

2.5 PairNorm 11

3 Experimental Methodology **13**

3.1 Real-World Datasets 14

3.2 Evaluation Protocol 14

3.3 Oversmoothing Diagnostics 15

4 Broad Real-World Screening **18**

4.1 Experimental Design 18

4.2 Results 19

5 Selected Real-World Cases **23**

5.1 Depth Interaction 23

5.2 Representation Diagnostics 26

6 Controlled Homophily **27**

6.1 Controlled Graph Construction 27

6.2 Endpoint Results 29

6.3 Temporal Representation Dynamics 32

7 Controlled cSBM Experiments **34**

7.1 Synthetic Graph and Feature Construction 34

7.2 Predictive Performance Across Controlled Conditions 35

7.3 Critical Depth and Representation Collapse 37

8 Learning-Rate Sensitivity **38**

9 PairNorm Component Ablation **40**

9.1 Ablation Results 40

9.2 Interpretation of the Components 44

10 Oversmoothing Synthesis **44**

1 Introduction

1.1 Motivation

1.2 Research Questions

2 Background

2.1 Message Passing and Receptive Fields

A graph can be represented as $G = (V, E)$, where V is the set of nodes and E is the set of edges connecting pairs of nodes. In the graph-learning setting considered in this thesis, each node $v \in V$ also has an input feature vector $\mathbf{x}_v$. A graph neural network (GNN) uses these node features and the graph structure to create learned representations of the nodes. A widely used class of GNNs follows the *message-passing* principle, in which information is exchanged between adjacent nodes over a sequence of neural-network layers [GSR+17].

Let $\mathbf{h}_v^{(l)}$ denote the representation of node v after layer l, where $\mathbf{h}_v^{(0)} = \mathbf{x}_v$. In a message-passing layer, node v first receives information from the neighboring nodes in $\mathcal{N}(v)$. Messages generated from neighboring nodes are then combined using an aggregation operation. The result is then used to update the representation of node v. Following the message-passing framework of Gilmer et al. [GSR+17], this process can be written in the form

$$\mathbf{m}_v^{(l+1)} = \mathrm{AGG}^{(l)} \left(\left\{ M^{(l)} \left(\mathbf{h}_v^{(l)}, \mathbf{h}_u^{(l)}, \mathbf{e}_{vu} \right) \,\middle|\, u \in \mathcal{N}(v) \right\} \right), \tag{1}$$

followed by

$$\mathbf{h}_v^{(l+1)} = U^{(l)} \left(\mathbf{h}_v^{(l)}, \mathbf{m}_v^{(l+1)} \right). \tag{2}$$

Here, $M^{(l)}$ is the message function of layer l, $\mathrm{AGG}^{(l)}$ combines the messages arriving from the neighbors of v, and $U^{(l)}$ updates the representation of the target node. The optional vector $\mathbf{e}_{vu}$ represents features associated with the edge between nodes v and u. Since the neighbors of a node do not have a fixed order, the aggregation should not depend on the order in which these neighbors are presented. Common examples are sum, mean or maximum aggregation. The way these functions are used may differ depending on the GNN architecture [GSR+17, HYL17].

One such GNN model is GraphSAGE, which is extensively used in the experiments of this thesis and provides a concrete example of this procedure. At each layer, GraphSAGE first aggregates information from the immediate neighborhood of a node. The aggregated information is then combined with the node's own representation to obtain an updated node representation [HYL17]. Since the neighborhood representations used at one layer were themselves constructed in the preceding layer, information can propagate recursively through the graph. Thus, the first message-passing layer can incorporate information from direct neighbors. The second layer can additionally incorporate information from two edges away, and this process continues with increasing depth.

This recursive expansion motivates the notion of a *receptive field*. For a target node, the theoretical receptive field includes all nodes whose input information may influence its representation after a given number of message-passing layers. An L-layer network in which messages are passed between directly

connected nodes can in principle incorporate information from nodes that are at most L edges away from the target node. Therefore, adding more message-passing layers allows information to travel over a larger graph distance. Xu et al. describe the same principle through neighborhood aggregation. After k iterations, the representation can use structural information rooted at the target node up to depth k [XLT+18]. Figure 1 illustrates this expansion for successive message-passing layers.

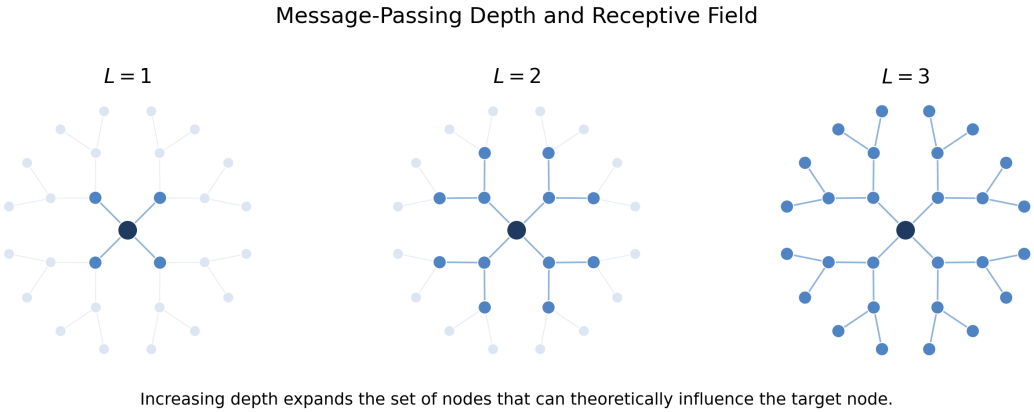


Figure 1: Receptive-field growth under repeated message passing. Each additional message-passing layer increases the maximum graph distance from which information can potentially reach the target node.

The theoretical receptive field should, however, be distinguished from the amount of information that actually influences a learned representation. For example, a node can be inside the L-hop neighborhood because a path to the target exists within L message-passing steps. However, this does not mean that all nodes in this neighborhood have the same influence. The learned parameters, aggregation operation, nonlinearities, graph structure and possible neighborhood sampling determine the strength of the influence from different nodes on the final representation. Xu et al. formalize this distinction by studying the sensitivity of a target representation to features at other nodes and show that the effective range of influence depends strongly on the local graph structure [XLT+18]. Therefore, two target nodes processed by networks of the same depth can have the same maximum hop range while exhibiting substantially different effective information propagation.

This distinction is central to the remainder of this thesis. Increasing depth provides a larger theoretical receptive field and therefore creates the possibility of incorporating information from more distant parts of a graph. It does not, by itself, guarantee that this information is preserved or used effectively. The following sections therefore separate the number of propagation steps from the capacity of the intermediate representations and subsequently examine how repeated message passing can affect the geometry and usefulness of node representations.

2.2 Depth versus Hidden Dimension

The previous section established that stacking message-passing layers increases the maximum graph distance over which information can potentially reach a target node. Network depth therefore determines how many propagation steps are possible. Nevertheless, the information that a graph neural network can represent is not determined by its depth alone. Another key architectural factor is the dimensionality of the intermediate node representations, often referred to as the *hidden dimension* or network width.

Using the notation from Section 2.1, let the representation of node v at layer l be $\mathbf{h}_v^{(l)}$. If the hidden representation contains p components, then

$$\mathbf{h}_v^{(l)} \in \mathbb{R}^p. \tag{3}$$

Network depth L and hidden dimension p describe two different aspects of the network. Depth specifies how many successive message-passing steps are applied. In architectures based on local one-hop message passing, greater depth means information can propagate over larger distances, thereby expanding the theoretical receptive field. Hidden dimension, on the other hand, specifies the size of the vector used to store and transform a node's intermediate representation. Therefore, increasing p does not add another propagation step, which means it does not extend the receptive field [Lou20].

This difference is illustrated in Figure 2. Increasing depth adds further message-passing layers, resulting in information passing through additional edges before the final representation is obtained. By contrast, increasing the hidden dimension enlarges the intermediate representation available at each layer. The two changes can therefore influence the model in different ways. Throughout this thesis, the term *hidden dimension* refers to the common dimensionality assigned to the intermediate hidden representations.

The distinction between these quantities is also relevant from an expressive-power perspective. Loukas studies message-passing graph neural networks under explicit restrictions on depth and width and shows that, for several graph problems in the considered theoretical setting, restricting these parameters limits the computational power of the network [Lou20]. The resulting bounds reveal a trade-off between depth and width for these problems, but they do not imply that the two architectural parameters are generally interchangeable. The theoretical analysis derives worst-case lower bounds under a defined computational model. Despite the identified trade-off, depth and width remain distinct architectural properties.

This difference is especially relevant when information from distant nodes must be preserved across several message-passing steps. Increasing the hidden dimension provides more space for intermediate representations but does not shorten the paths along which information travels. Conversely, increasing depth enables the network to gather information from nodes that are further away without changing the size of the hidden representations. Di Giovanni et al. analyze these effects separately in the context of long-range information propagation and show that increasing width and increasing depth can have fundamentally different effects on the sensitivity of message-passing neural networks to distant node information [DGGB$^+$23]. Their findings also highlight that increasing network depth cannot generally compensate for a limited representation width. Therefore, when comparing shallow and deep graph neural networks, it is important to distinguish between the impact of additional message-passing steps and changes in the dimensionality of node representations.

This thesis follows that distinction throughout the empirical analysis. Network depth is varied to investigate the effects of successive message-passing steps, whereas the hidden dimension is recorded or controlled independently. Separating these two architectural factors is necessary for understanding whether performance differences in deeper networks can be attributed to graph structure, representation geometry, or other characteristics of the learning task.

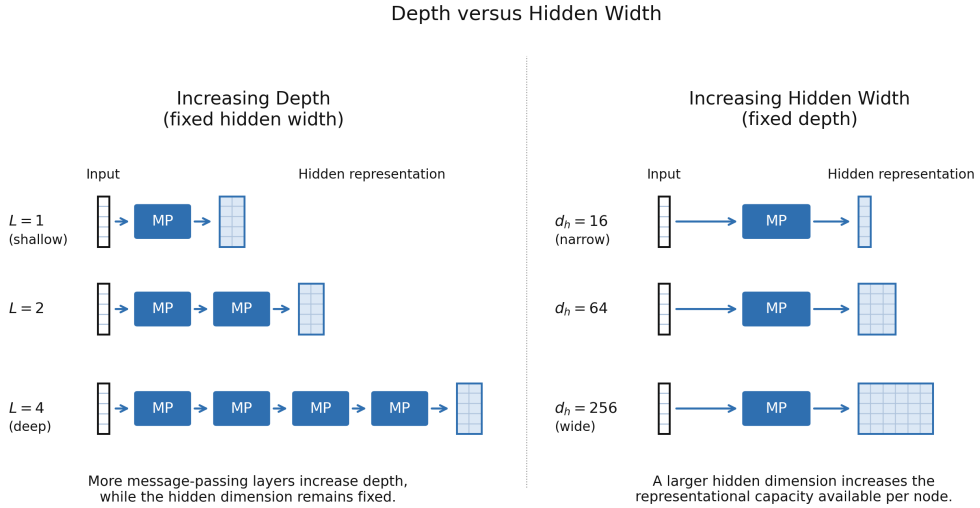


Figure 2: Conceptual distinction between depth and hidden dimension in a message-passing graph neural network. Increasing depth adds successive propagation steps and expands the theoretical receptive field, whereas increasing the hidden dimension enlarges the intermediate node representation without adding another message-passing step.

2.3 Homophily and Heterophily

The previous sections provided an overview of how message passing enables information to propagate between nodes in a graph and how network depth and hidden dimension influence different aspects of the GNN architecture. However, the usefulness of information aggregated from neighboring nodes also depends on the relation between the graph topology and the learning task. In node classification, this relation is often described in terms of *homophily* and *heterophily*.

In general, homophily describes a tendency of connected nodes to be similar, whereas heterophily describes a tendency of connected nodes to be dissimilar. The notion of similarity must be specified explicitly, since it may refer to class labels, node features, or other node attributes. In the graph-learning literature and in the experiments of this thesis, the term is primarily used with respect to class labels [ZYZ+20, ARE26]. Accordingly, a graph is more class-homophilous if connections frequently occur between nodes of the same class. In contrast, it is more class-heterophilous when edges frequently connect nodes belonging to different classes.

A simple measure of this property is the *edge homophily ratio*. For a graph $G = (V, E)$ with class label y_v assigned to each node v, it is defined as

$$h_{\text{edge}} = \frac{|\{(u, v) \in E \mid y_u = y_v\}|}{|E|}. \tag{4}$$

The value h_{edge} therefore represents the fraction of edges whose endpoints have the same class label [ZYZ+20, PKD+23]. Values close to one indicate that most edges are within classes, while values close to zero indicate that most edges connect nodes from different classes. Unless stated otherwise, the symbol h in this thesis refers to this raw class-label edge homophily measure.

The relation to message passing becomes intuitive when considering the aggregation mechanism introduced in Section 2.1. In a strongly homophilous graph, nodes typically aggregate information from neighbors that share the same class label. If their features also contain class-relevant information, the aggregation process can strengthen the signals relevant to the classification task. In a heterophilous graph, nodes often have neighbors belonging to different classes. Depending on the node features and task conditions, directly mixing these representations can make class-relevant information harder to separate. This observation helps explain why many graph neural network architectures and benchmarks have historically been associated with a homophily assumption and why specialized methods have been developed to address heterophilous graphs [ZYZ+20].

Figure 3 illustrates this distinction at the level of class labels. In the homophilous example, most edges connect nodes with the same class, whereas the heterophilous example contains a larger proportion of cross-class edges. The figure should be interpreted as an illustration of label–topology alignment rather than as a statement about task difficulty. In particular, the fact that neighboring nodes often belong to different classes does not imply that their relation is uninformative.

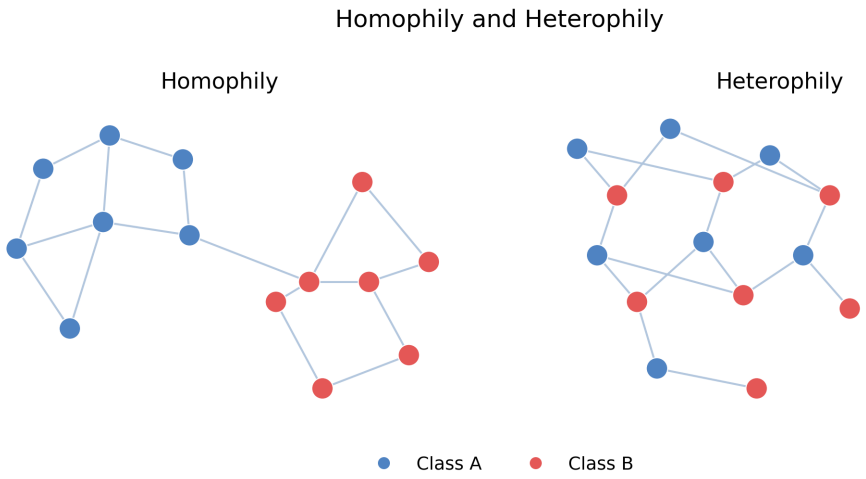


Figure 3: Schematic comparison of class-homophilous and class-heterophilous graph structure. In the homophilous example, edges predominantly connect nodes of the same class, whereas the heterophilous example contains a larger proportion of edges between different classes. The schematic illustrates label–topology alignment and does not imply that one of the two settings is inherently easier to learn.

This distinction is important because homophily describes the relationship between graph structure and the learning task rather than directly measuring how well a graph neural network will perform. Empirical studies have shown that standard GNNs can achieve strong predictive performance on some heterophilous graphs, while high homophily alone does not guarantee that a particular node-classification task is easy [PKD+23, ARE26]. The usefulness of neighborhood aggregation depends on additional factors, including the node features, the relationship between features and labels, the specific connectivity pattern, the learning objective, and the architecture used to process the graph.

A further limitation is that raw edge homophily should not be treated as an absolute measure that is directly comparable across arbitrary datasets. In particular, the fraction of same-class edges is affected by the number and relative sizes of the classes. Platonov et al. therefore distinguish raw edge homophily from adjusted measures that account for the amount of same-class connectivity expected from the class and degree distribution [PKD+23]. This thesis nevertheless uses raw edge homophily when a direct and

interpretable fraction of same-label edges is required, especially in the controlled experiments where target values of h are imposed explicitly. Adjusted homophily is used as a supporting diagnostic where appropriate rather than as a replacement for the controlled variable.

In this thesis, homophily and heterophily are therefore used to describe how labels align with graph structure, not as a sufficient explanation for model behavior. Arnaiz-Rodriguez and Errica similarly caution against the universal claim that homophily is inherently beneficial and heterophily inherently detrimental, emphasizing instead the interaction between the task, features, labels, and topology [ARE26]. This distinction is particularly relevant for the experiments presented later in this thesis, where homophily is varied under controlled conditions to examine whether differences in label–topology alignment are associated with changes in depth-related performance degradation and learned node representations.

2.4 Oversmoothing

Repeated message passing does not only enlarge the receptive field of a node. It also repeatedly mixes node representations with information from their neighborhoods. As discussed in the previous section, this mixing can be useful when neighboring nodes contain task-relevant information. However, repeated aggregation can cause node representations to become gradually similar across multiple layers. This phenomenon is commonly referred to as *oversmoothing* [LHW18, RBM23].

An early explanation of this effect was given by Li et al. [LHW18] for graph convolutional networks. They show that the propagation operation of a graph convolutional network can be understood as a form of Laplacian smoothing. Under this interpretation, each propagation operation combines a node's own representation with information from its neighbors. A moderate amount of such smoothing can be beneficial, as it encourages related nodes to develop more similar representations. On the other hand, excessive smoothing can eventually remove distinctions that are necessary for separating nodes, making it difficult to distinguish between them. Oversmoothing therefore refers to excessive representation smoothing rather than to neighborhood aggregation itself.

Let

$$H^{(l)} = \begin{bmatrix} (\mathbf{h}_1^{(l)})^\top \\ \vdots \\ (\mathbf{h}_{|V|}^{(l)})^\top \end{bmatrix}$$

denote the matrix containing all node representations at layer l. One way to measure how much the representations of neighboring nodes differ is through the graph Dirichlet energy. In a simplified unnormalized form, it can be written as

$$E\Big(H^{(l)}\Big) = \frac{1}{2} \sum_{(u,v)\in E} \left\| \mathbf{h}_u^{(l)} - \mathbf{h}_v^{(l)} \right\|_2^2. \tag{5}$$

Higher values indicate greater differences between the representations of neighboring nodes, whereas a value approaching zero indicates that representations connected by graph edges are becoming increasingly similar. Therefore, measures based on Dirichlet energy are widely used as diagnostics of oversmoothing [RBM23]. The exact normalized Dirichlet energy used in the experiments of this thesis is defined in a later section, since normalization is important when representation magnitudes differ across models or training conditions.

Several definitions and metrics for oversmoothing have been proposed in the literature. Rusch et al. [RBM23] provide a unified formulation in which oversmoothing is characterized by the layer-wise ex-

ponential convergence of a suitable node-similarity measure toward its collapsed value. This definition emphasizes that oversmoothing is not merely the observation that two neighboring representations happen to become slightly more similar. Rather, it describes a systematic loss of representation diversity as network depth increases. Other analyses characterize the limiting representations differently depending on the propagation operator. For example, Oono and Suzuki demonstrate that, under specific conditions on the graph spectrum and weight matrices, the representations in deep graph convolutional networks converge exponentially toward a restricted subspace [OS20]. In this subspace, the remaining information can be insufficient to differentiate between nodes that belong to the same connected component and have the same degree.

While these asymptotic results help explain why oversmoothing can occur, they do not suggest that every deep graph neural network will experience oversmoothing in the same way. The behavior depends on the propagation operator, graph topology, parameterization, normalization, and training dynamics. Recent work has further demonstrated that different oversmoothing diagnostics can lead to different conclusions for the same representations. In particular, Arnaiz-Rodriguez and Errica demonstrate that the raw Dirichlet energy can vary significantly due to simple rescaling, even when the separability of node embeddings remains largely unchanged [ARE26]. This motivates the use of multiple, scale-aware representation diagnostics rather than relying on a single metric.

Figure 4 illustrates the qualitative idea considered in this thesis. Node representations that are initially separated can move closer together as neighborhood information is repeatedly aggregated. The figure is meant to provide a simplified illustration of how node representations can become less diverse. It does not suggest that this process occurs in the same way for every graph, network architecture, or training setup.

A further distinction is essential for the empirical analysis in this thesis. Oversmoothing describes a change in the geometry or information content of node representations, but oversmoothing itself is not equivalent to a decrease in predictive accuracy. If all node representations become indistinguishable and lose their class-relevant information, a classifier can no longer distinguish between nodes that require different predictions. However, before complete representation collapse occurs, smoothing may reduce irrelevant differences between node representations while maintaining or even enhancing their class-related separability. Conversely, predictive performance can decrease for reasons other than oversmoothing, including optimization difficulties, an uninformative receptive field, or a mismatch between the graph structure and the learning task [KDL26, ARE26].

This distinction has received increasing attention in recent work. Kormann et al. report that methods designed to mitigate oversmoothing can preserve representation diversity without consistently producing corresponding improvements in predictive performance [KDL26]. Arnaiz-Rodriguez and Errica similarly argue that performance degradation should not be attributed to oversmoothing by default and emphasize that task-relevant representation separability is more directly linked to predictive performance [ARE26]. For this reason, the experiments in this thesis evaluate predictive performance and representation diagnostics separately. Evidence of representation collapse is considered an indication of oversmoothing only when it is considered together with the corresponding model performance.

Since repeated message passing can reduce important differences between node representations, several methods aim to preserve or restore their geometric structure. One such method is PairNorm, which was proposed specifically as a normalization-based approach for counteracting oversmoothing and is introduced in the following section.

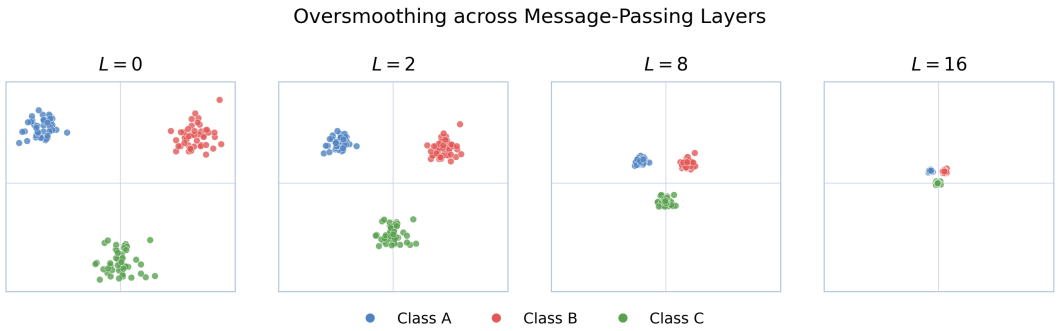


Figure 4: Conceptual illustration of oversmoothing under repeated message passing. Initially distinct node representations can become progressively more similar as neighborhood information is repeatedly aggregated. The schematic illustrates representation collapse and does not imply that all graph neural networks oversmooth at the same depth or rate.

2.5 PairNorm

The previous section described oversmoothing as a progressive loss of distinction between node representations under repeated message passing. This observation motivates normalization methods that do not prevent neighborhood aggregation itself, but constrain how strongly the geometry of the intermediate representations can contract. PairNorm is one such method. It was introduced by Zhao and Akoglu as a normalization layer specifically designed to counteract oversmoothing in graph neural networks [ZA20].

The main idea of PairNorm is to control the global spread of the node representations after a message-passing layer. Let $\widetilde{H} \in \mathbb{R}^{n \times p}$ denote the matrix of intermediate representations produced by such a layer, where n is the number of nodes and p the hidden dimension. Its i-th row, $\widetilde{\mathbf{h}}_i$, represents node i. Zhao and Akoglu motivate PairNorm through the observation that repeated graph convolution tends to decrease differences between node representations. They therefore propose constraining the total pairwise squared distance

$$\mathrm{TPSD}(H) = \sum_{i=1}^{n} \sum_{j=1}^{n} \|\mathbf{h}_i - \mathbf{h}_j\|_2^2 , \tag{6}$$

where TPSD denotes the total pairwise squared distance [ZA20]. Maintaining a non-vanishing global pairwise distance prevents all node representations from simultaneously collapsing toward the same point.

PairNorm implements this idea using two consecutive operations: *centering* and *scaling*. First, the mean representation across all nodes is removed. For each node i, the centered representation is

$$\widetilde{\mathbf{h}}_i^c = \widetilde{\mathbf{h}}_i - \frac{1}{n} \sum_{j=1}^{n} \widetilde{\mathbf{h}}_j . \tag{7}$$

This translation moves the mean of the node representations to the origin. Since the same vector is subtracted from every node, centering does not change the pairwise differences between node representations. Its purpose is instead to provide a common reference point from which the overall scale of the representation matrix can be controlled efficiently [ZA20].

The centered representations are then rescaled according to their average squared norm. The standard PairNorm transformation can be written as

$$\mathbf{h}_i^{\mathrm{PN}} = s \frac{\widetilde{\mathbf{h}}_i^c}{\sqrt{\frac{1}{n}\sum_{j=1}^{n} \left\| \widetilde{\mathbf{h}}_j^c \right\|_2^2}}, \tag{8}$$

where $s > 0$ is a scale hyperparameter [ZA20]. After this transformation, the representations remain centered and their total pairwise squared distance is fixed to a value determined by n and s. In particular, the original PairNorm derivation gives

$$\mathrm{TPSD}\left(H^{\mathrm{PN}}\right) = 2n^2 s^2. \tag{9}$$

PairNorm therefore does not prescribe which individual nodes should be close to or distant from one another. Instead, it constrains the global geometry of the complete set of node representations. Message passing can still make some representations more similar, but a simultaneous collapse of all node representations is prevented by maintaining their overall spread. This is important because PairNorm does not modify the graph topology, the receptive field, or the number of message-passing steps. It only acts on the intermediate representations produced by the network.

This geometric effect is illustrated in Figure 5. Repeated aggregation can initially move distinct node representations toward a more concentrated region of the embedding space. PairNorm first recenters these representations and after that it rescales their global spread. The figure should not be understood as suggesting that all distances between nodes are preserved or that their original positions are recovered. Rather, it visualizes the global constraint that prevents the complete representation set from contracting without bound.

The original formulation is computationally simple and introduces no additional learnable parameters. Zhao and Akoglu propose inserting the normalization between intermediate graph neural network layers, which allows PairNorm to be combined with different message-passing architectures [ZA20]. In this thesis, PairNorm is primarily combined with GraphSAGE so that the effect of repeated message passing can be compared with and without this normalization under otherwise matched architectural conditions. The exact model placement and experimental settings are specified later in the methodology.

However, it should be said that maintaining diversity among node representations does not necessarily lead to higher predictive accuracy. Zhao and Akoglu already observe that increased robustness against oversmoothing in deep graph neural networks does not necessarily improve absolute test performance when additional depth is not useful for the underlying task [ZA20]. Rusch et al. further demonstrate that a method can maintain a non-collapsed representation diagnostic, even as predictive performance declines with increasing network depth [RBM23]. PairNorm is therefore considered in this thesis as a method for modifying the geometry of node representations rather than a guaranteed solution to depth-related performance degradation.

This distinction also motivates a later component ablation. Since the standard PairNorm transformation consists of both centering and scaling, the experiments additionally evaluate a center-only and a scale-only operation to determine which component is associated with the observed changes in representation geometry and predictive performance. These two configurations are introduced specifically for the analysis in this thesis and are not presented as distinct methods in the original PairNorm work.

Finally, PairNorm is also reused later in the long-range information propagation experiments, where it continues to function as a non-structural normalization mechanism. Unlike graph rewiring, PairNorm does not introduce new edges or shorten graph distances. This makes it possible to examine whether

changes in long-range behavior necessarily require a structural modification of the graph, without redefining PairNorm as an oversquashing-specific method.

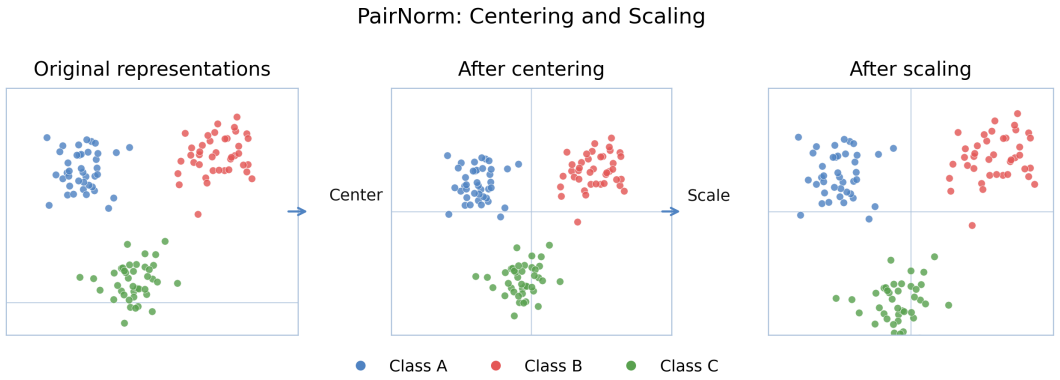


Figure 5: Conceptual effect of PairNorm on node-representation geometry. After message passing has reduced the global spread of the representations, PairNorm recenters them and rescales the representation matrix to a fixed global pairwise-distance scale. The operation constrains the overall geometry and does not preserve each individual pairwise distance.

3 Experimental Methodology

The previous chapter introduced the theoretical concepts needed to understand why graph neural networks may perform worse as their depth increases. In particular, it distinguished the loss of representation diversity associated with oversmoothing from a decrease in predictive accuracy. The empirical analysis follows this distinction by evaluating both aspects separately and examining whether they change together under controlled modifications of network depth, normalization, and graph structure.

The experiments are divided into several stages that complement each other. The first stage consists of a broad screening study that investigates the performance of shallow and deeper graph neural networks across 15 real-world node-classification datasets. Out of the 15 datasets, four are then selected for a more detailed comparison of GraphSAGE with and without PairNorm. To make the comparisons more controlled, additional experiments change the level of edge homophily in PubMed and Roman-Empire while preserving their original node features and labels. A synthetic contextual stochastic block model provides further control over graph connectivity and feature information. Finally, learning-rate sensitivity and component-ablation experiments investigate whether the observed differences depend on optimization or on particular operations of PairNorm.

These stages serve different purposes. The broad screening study examines how predictive performance changes with increasing network depth and how these effects differ between datasets. The subsequent controlled experiments provide further insight into how node representations, model performance, and normalization are related. However, no measurement on its own is considered sufficient to conclude that oversmoothing is responsible for an observed decline in accuracy.

3.1 Real-World Datasets

As outlined in the previous section, the real-world evaluation consists of 15 node-classification datasets that differ in graph size, feature dimensionality, number of classes, and degree of class-label homophily. Node classification is the task of predicting a class label for each node using the available node features and graph structure. In the experiments considered here, the graph and node features are available during message passing, while only the labels of the training nodes are used to optimize the model parameters.

Table 1 summarizes the dataset characteristics. The number of edges in the table refers to unique undirected node pairs. The feature count denotes the dimensionality of the input feature vector, and the number of classes specifies the possible prediction labels. The edge-homophily values use the raw class-label definition introduced in Section 2.3. The table also contains the dataset loader and corresponding source for each employed dataset version.

The dataset collection deliberately includes both relatively homophilous and strongly heterophilous graphs to cover a broad range of graph characteristics. The datasets also differ substantially in size. This diversity is useful for evaluating whether a pattern associated with increasing network depth is specific to one benchmark or appears across substantially different graph-learning conditions.

Table 1: Overview of the 15 real-world node-classification datasets. The source column identifies the benchmark reference for the employed dataset version.

Dataset	Nodes	Edges	Features	Classes	h	Loader	Source
Cora	2,708	5,278	1,433	7	0.810	Planetoid	[YCS16]
CiteSeer	3,327	4,552	3,703	6	0.736	Planetoid	[YCS16]
PubMed	19,717	44,324	500	3	0.802	Planetoid	[YCS16]
Roman-Empire	22,662	32,927	300	18	0.047	HeterophilousGraphDataset	[PKD$^+$23]
Amazon-Photo	7,650	119,081	745	8	0.827	Amazon	[SMBG18]
Amazon-Computers	13,752	245,861	767	10	0.777	Amazon	[SMBG18]
Coauthor-CS	18,333	81,894	6,805	15	0.808	Coauthor	[SMBG18]
Coauthor-Physics	34,493	247,962	8,415	5	0.931	Coauthor	[SMBG18]
Actor	7,600	26,752	932	5	0.219	Actor	[PWC$^+$20]
Chameleon	2,277	31,421	2,325	5	0.235	WikipediaNetwork	[PWC$^+$20, RAS21]
Squirrel	5,201	198,493	2,089	5	0.224	WikipediaNetwork	[PWC$^+$20, RAS21]
Cornell	183	280	1,703	5	0.131	WebKB	[PWC$^+$20]
Texas	183	295	1,703	5	0.108	WebKB	[PWC$^+$20]
Wisconsin	251	466	1,703	5	0.196	WebKB	[PWC$^+$20]
Amazon-Ratings	24,492	93,050	300	5	0.380	HeterophilousGraphDataset	[PKD$^+$23]

Note. Features denotes the input feature dimensionality. Classes denotes the number of target classes. h is raw class-label edge homophily. Loader identifies the dataset class used in PyTorch Geometric.

3.2 Evaluation Protocol

Throughout the experiments, model training, model selection, and final performance evaluation are treated as separate stages. This separation prevents test results from influencing the choice of model configurations, which could otherwise lead to biased performance estimates.

In each node-classification experiment, labeled nodes are divided into training, validation, and test sets following the respective experimental protocol. Training labels are used to update the model parameters, while validation labels are used to select checkpoints and, where applicable, model configurations and

hyperparameters. Test labels are used exclusively to evaluate the predictive performance of the selected model and do not influence model selection.

The main predictive measure is test accuracy, which represents the proportion of test nodes that are correctly classified. It is calculated as

$$\mathrm{Acc}_{\mathrm{test}} = \frac{N_{\mathrm{correct}}}{N_{\mathrm{test}}}, \tag{10}$$

where N_{correct} is the number of correctly classified test nodes and N_{test} is the total number of test nodes. When accuracy is expressed as a percentage, the result is then multiplied by 100.

To select the checkpoint, validation accuracy is evaluated during training. The model checkpoint with the highest validation accuracy is selected, and the test accuracy reported for that run corresponds to this checkpoint. As a result, achieving high training accuracy alone is not sufficient for selecting a model, especially if it performs poorly on the validation nodes.

Two kinds of model comparison are used. In the initial screening study, model families and hyperparameters are selected according to their validation performance within predefined candidate sets. While these comparisons characterize the predictive performance of the selected configurations, they do not isolate the contribution of individual architectural modifications. In the subsequent experiments, model configurations are matched wherever possible to investigate differences associated with network depth and PairNorm under comparable conditions.

To account for variability between experimental runs, the experiments use repeated evaluations with different random seeds or data splits, depending on the experimental setup. In the controlled experiments, graph seeds determine the generated or rewired graph instances, while initialization seeds control model initialization and training randomness. It is important to distinguish between these sources of variability because training models with different initializations on the same graph does not produce independent graph instances. When experiments involve multiple graph seeds and model initialization seeds, the results are first averaged across initializations for each graph seed. These graph-level averages are then used to summarize performance and variability across the different graph instances. Details on the number of repetitions and the aggregation procedure are provided in the respective experimental sections.

Another important consideration concerns the evaluation of node representations. While predictive accuracy is assessed using the checkpoint selected based on validation performance, certain representation metrics are measured at a predefined training epoch. As a result, these measurements reflect the geometry of node representations at that specific point in training and may differ from those of the checkpoint used for test evaluation. This difference is considered when analyzing the relationship between representation collapse and predictive performance.

Finally, the experiments were conducted using different training implementations and configurations. In particular, one of the real-world experimental setups uses a normalization procedure that differs from the standard PairNorm scaling introduced in Section 2.5 [ZA20], whereas the cSBM experiments use the implementation provided by PyTorch Geometric. For this reason, results are primarily compared within their respective experimental settings, and differences across experiment groups are not attributed solely to PairNorm.

3.3 Oversmoothing Diagnostics

A decline in classification accuracy does not establish whether node representations have become less distinguishable. Oversmoothing is therefore investigated using representation-level measurements in ad-

dition to predictive performance.

The primary diagnostic measures used in these experiments are normalized Dirichlet energy (NDE) and effective rank. Pairwise and edge-based cosine distances are used as supporting measurements. These quantities describe different geometric properties of the representations and should not be interpreted as interchangeable estimates of predictive quality. This multi-metric approach is motivated by the fact that oversmoothing measures can capture distinct properties of node representations and may therefore lead to different conclusions depending on the experimental setting [RBM23, ARE26, ZDHT26].

The analysis focuses on intermediate node representations rather than predicted class logits. In the main experiments, the relevant hidden representation has 128 dimensions and is taken from the final hidden stage before the classifier. The exact point at which these representations are recorded may vary depending on the model implementation. For instance, outputs from message-passing and normalization operations may be captured at different stages relative to activation functions or dropout. Therefore, the recorded diagnostic values are interpreted with respect to the specific representation stage from which they were obtained, rather than being treated as representative of all intermediate operations.

Normalized Dirichlet energy. As introduced in Section 2.4, Dirichlet energy measures differences between the representations of neighboring nodes [RBM23]. The unnormalized form was defined in Equation (5). However, its value depends on the magnitude of the node representations. Multiplying all representations by the same factor changes the Dirichlet energy without changing their relative geometric arrangement. This makes direct comparisons between models or training conditions difficult when representation magnitudes differ [ARE26].

To address this limitation, the experiments use normalized Dirichlet energy (NDE). Instead of considering only the squared differences between neighboring nodes, NDE compares the average squared difference across edges with the average squared norm of the node representations. For an undirected graph $G = (V, E)$ and a representation matrix H, it is calculated as

$$\mathrm{NDE}(H) = \frac{\frac{1}{|E|}\sum_{\{u,v\}\in E} \|\mathbf{h}_u - \mathbf{h}_v\|_2^2}{\frac{1}{|V|}\sum_{v\in V} \|\mathbf{h}_v\|_2^2 + \varepsilon}, \tag{11}$$

where $|E|$ is the number of undirected edges, $|V|$ is the number of nodes and ε is a small constant used for numerical stability. Unlike the unnormalized Dirichlet energy introduced in Section 2.4, the numerator represents the average squared distance between connected node representations, while the denominator accounts for the overall magnitude of the node representations.

When the denominator is sufficiently large relative to ε, NDE remains approximately unchanged when node representations are uniformly rescaled. Values approaching zero indicate that neighboring nodes have very similar representations relative to their overall magnitude, whereas larger values indicate greater differences between connected node representations relative to their magnitude.

However, NDE measures differences between connected nodes rather than how well representations separate different classes. A higher NDE therefore does not necessarily indicate better predictive performance. Depending on the graph structure and the classification task, greater variation between neighboring nodes may not correspond to more useful class information [RBM23, ARE26].

Effective rank. Normalized Dirichlet energy describes variation across edges, but it does not directly measure how many independent directions are represented in the hidden feature matrix. To characterize this second aspect, the experiments use effective rank.

Let $\sigma_1, \ldots, \sigma_q$ denote the singular values of a representation matrix H, where $q = \min(|V|, p)$ and p is the hidden dimension. For a nonzero singular-value sum, define

$$w_k = \frac{\sigma_k}{\sum_{j=1}^{q} \sigma_j}. \tag{12}$$

Following the entropy-based definition introduced by Roy and Vetterli [RV07], effective rank is

$$\operatorname{erank}(H) = \exp\left(-\sum_{k=1}^{q} w_k \log w_k\right), \tag{13}$$

using the convention $0 \log 0 = 0$.

When one singular value dominates the spectrum of a representation matrix, its effective rank approaches one. In contrast, a higher effective rank indicates that variation is spread across multiple independent directions. Unlike the standard matrix rank, which only considers whether singular values are nonzero, the effective rank also takes their relative magnitudes into account. For zero matrices, a numerical convention is required because the normalized singular-value weights are undefined.

For comparisons across hidden dimensions, the experiments additionally use the effective-rank ratio

$$r_{\text{eff}}(H) = \frac{\operatorname{erank}(H)}{p}. \tag{14}$$

This normalized measure describes the effective dimensionality of the representations relative to the available hidden dimension. When the hidden dimension is held constant, both effective rank and the normalized ratio yield the same ranking of models.

Effective rank provides a complementary perspective to Dirichlet energy. Node representations can exhibit considerable differences across graph edges even when most of their variation is concentrated in a low-dimensional subspace. On the other hand, representations can be distributed across several directions without providing useful information for distinguishing between classes. Therefore, a higher effective rank does not necessarily indicate better predictive performance. Zhang et al. have proposed rank-based measures to address limitations of energy-based oversmoothing diagnostics [ZDHT26].

Cosine-distance diagnostics. In addition to NDE and effective rank, the experiments use cosine-based distances to examine the similarity between node representations. Unlike Euclidean distance, cosine distance focuses on the direction of two vectors rather than their magnitude. The Mean Average Distance (MAD) formulation discussed by Rusch et al. includes cosine distance as one of its components [RBM23]. For two nonzero node representations, cosine distance is defined as

$$d_{\cos}(\mathbf{h}_u, \mathbf{h}_v) = 1 - \frac{\mathbf{h}_u^\top \mathbf{h}_v}{\|\mathbf{h}_u\|_2 \|\mathbf{h}_v\|_2}, \tag{15}$$

where $\mathbf{h}_u$ and $\mathbf{h}_v$ are the representations of two nodes. The numerator is their dot product, while the denominator is the product of their Euclidean norms. Together, these quantities describe how closely the two vectors point in the same direction. A cosine distance close to zero indicates that the representations point in nearly the same direction, whereas larger values indicate greater differences in their directions.

The experiments consider cosine distances between node pairs and separately between nodes connected by an edge. These measurements complement NDE because they describe directional similarity indepen-

dently of the absolute magnitudes of the representations. However, cosine distance can become numerically unstable when the norm of a representation is close to zero. It also does not measure differences in vector magnitude or determine whether the representations contain useful information for classification.

PairNorm can also influence cosine distances by changing the directions of node representations through its centering operation. For these reasons, cosine distances are used only as supporting measurements and are interpreted together with predictive accuracy, training dynamics, and the other representation diagnostics.

Interpretation of diagnostic evidence. The subsequent experiments therefore use three distinct types of evidence. Predictive accuracy is used to assess classification performance, while representation diagnostics measure properties of the learned hidden vectors and matched interventions examine how both quantities respond when depth, normalization, graph topology, or optimization settings change.

When a deep model exhibits both a severe accuracy decrease and a strong reduction in representation diversity, this provides evidence consistent with an oversmoothing-related failure. If an intervention preserves representation diversity and improves accuracy in the same controlled condition, the evidence becomes stronger. Nevertheless, such co-occurrence does not establish that the measured diagnostic causally mediates the accuracy improvement. PairNorm can also change optimization dynamics, and graph interventions can alter the information available to the model.

Conversely, a high NDE or effective rank does not guarantee accurate predictions. This distinction is essential for identifying the conditions in which oversmoothing is a useful explanation for deep-GNN degradation and those in which additional factors must be considered.

4 Broad Real-World Screening

4.1 Experimental Design

The first empirical study examines how predictive performance changes with message-passing depth across the 15 real-world datasets introduced in Section 3.1. Its purpose is to determine whether deeper graph neural networks exhibit a consistent performance decline or whether the effect depends on the dataset and model configuration. This broad comparison also identifies representative conditions for the more controlled experiments that follow.

The screening was divided into two phases. The first phase considered relatively shallow networks with $L \in \{2, 4\}$ message-passing layers. The second phase evaluated deeper networks with $L \in \{8, 16, 32\}$. Regarding hidden dimensions, both phases considered 64 and 128. The model families included GCN, GAT, and GraphSAGE, together with their corresponding PairNorm configurations.

The two-phase design separates two related questions. The comparison between shallow and deeper phases indicates how the predictive performance of validation-selected configurations changes when the candidate depth range increases. Within each phase, the baseline-only search is compared with an expanded search that also includes PairNorm configurations. This comparison examines whether allowing PairNorm candidates changes the selected predictive performance. However, since the selected configurations may differ in architecture, hidden dimension, and network depth, these comparisons do not isolate the effect of PairNorm alone.

Model and hyperparameter selection were performed using validation accuracy. For each dataset and comparison category, the configuration with the highest mean validation performance was selected from

the evaluated candidates. Ties in mean validation accuracy were resolved by preferring fewer layers, followed by a smaller hidden dimension and alphabetical model order. The reported test accuracy corresponds to the validation-selected configuration and its best-validation checkpoints. Repetitions were used to characterize variability where repeated observations were available.

The results are summarized in Table 2, in which the baseline columns consider only models without PairNorm, whereas the full columns also include configurations that incorporate PairNorm.

4.2 Results

The broad screening shows that increasing the permitted depth range does not affect all datasets in the same way. Several datasets exhibit substantial degradation when the selected configuration changes from the shallow to the deeper phase, whereas others maintain relatively stable results. These differences are illustrated in Figure 6.

One of the clearest examples of the differences between the shallow and deeper phases is Amazon-Computers. As described above, the configurations were selected based on their mean validation accuracy within each comparison category. For Amazon-Computers, the selected shallow baseline is a two-layer GCN with 128 hidden dimensions and a test accuracy of $83.71 \pm 0.79\%$. In the deeper phase, the selected baseline is an eight-layer GCN with 64 hidden dimensions and reaches only $56.63 \pm 9.74\%$. The difference between the phase means is approximately 27.08 percentage points. In addition to the large decrease in mean accuracy, the deeper baseline exhibits substantially greater variability. The result demonstrates that allowing greater depth does not necessarily improve predictive performance, even when the configuration is selected by validation accuracy.

The full model search produces a different result on the same dataset. In the deeper phase, GraphSAGE with PairNorm, eight layers, and 128 hidden dimensions reaches $82.37 \pm 1.47\%$. This is 25.74 percentage points above the selected deeper baseline, and it approaches the shallow performance level. Nevertheless, the selected deeper baseline is a GCN rather than GraphSAGE. Therefore, this comparison combines changes in architecture, hidden dimension, and normalization. It demonstrates the usefulness of including alternative deep configurations in the search, but it does not isolate a causal 25.74-percentage-point benefit from PairNorm.

Roman-Empire provides another pronounced depth-sensitive case. The selected shallow baseline, a four-layer GraphSAGE model, reaches $79.37 \pm 0.58\%$, whereas the selected deeper baseline, an eight-layer GraphSAGE model, reaches $68.06 \pm 0.67\%$. The decrease of approximately 11.31 percentage points occurs while the selected hidden dimension remains 128. Unlike the Amazon-Computers comparison, the model family is also unchanged, making this a more direct descriptive indication of depth-related degradation under the selected configurations. Allowing PairNorm in the deeper phase raises the selected accuracy to $71.62 \pm 0.78\%$, but the resulting performance remains noticeably below the shallow configuration.

Actor exhibits a similar contrast, although with substantially lower absolute accuracy. Its shallow baseline reaches $34.49 \pm 1.11\%$, while the selected deeper baseline reaches $26.40 \pm 1.25\%$. This corresponds to a decrease of 8.09 percentage points. The deeper full search instead selects an eight-layer GraphSAGE+PairNorm model, reaching $32.87 \pm 0.60\%$. Thus, the full search recovers much of the selected baseline's depth-related loss, but it does not exceed the shallow result.

Not every dataset follows this pattern. Coauthor-Physics retains comparatively stable performance, with selected baseline accuracy changing from $93.20 \pm 0.54\%$ in the shallow phase to $92.30 \pm 0.56\%$ in the deeper phase. The difference is approximately 0.90 percentage points. Similarly, PubMed changes from

77.20% for the selected shallow baseline to $77.57 \pm 1.38\%$ for the selected deeper baseline, showing no corresponding depth-related deterioration in this screening comparison. These cases demonstrate why degradation cannot be regarded as an inevitable consequence of increasing the number of message-passing layers.

The smaller WebKB datasets show further variability. Cornell changes from $71.08 \pm 5.56\%$ for the selected shallow baseline to $56.22 \pm 6.96\%$ for the deeper baseline. Texas decreases from $78.65 \pm 5.76\%$ to $65.68 \pm 7.21\%$, while Wisconsin decreases from $80.20 \pm 4.66\%$ to $66.67 \pm 7.22\%$. These substantial mean differences occur alongside relatively large standard deviations, emphasizing that individual dataset splits or random repetitions can influence the observed outcome.

Figure 6 highlights two main observations. First, the absolute difficulty and achievable accuracy differ considerably across datasets. Second, the magnitude of the change between the shallow and deeper phases is not simply proportional to the initial performance level. Comparing these results with the edge-homophily values in Table 1 also suggests that raw edge homophily alone does not explain the observed differences.

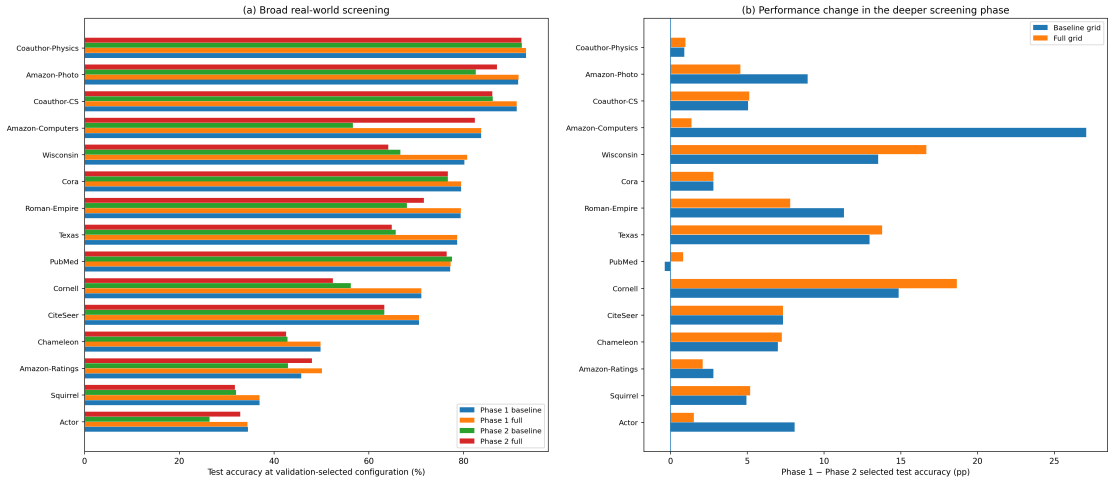


Figure 6: Broad real-world screening under validation-selected configurations. The left panel compares predictive performance across shallow and deeper phases, while the right panel summarizes the change between phases. Differences describe the selected configurations and do not isolate depth independently of all other hyperparameters.

In contrast to Figure 6, which examines differences between the shallow and deeper phases, Figure 7 focuses on the role of PairNorm in predictive performance. The analysis examines whether expanding the search to include PairNorm candidates improves the selected performance and whether consistent accuracy differences are observed when model configurations are matched.

The results differ across datasets and comparison protocols. For example, the large improvement of the selected full configuration on Amazon-Computers indicates that the expanded search finds a much stronger deep model, but part of this difference can arise from architecture selection. By contrast, matched comparisons are designed to hold the architecture and relevant hyperparameters fixed so that the remaining variation is more directly associated with normalization.

The Amazon-Ratings results also illustrate the importance of keeping these comparisons separate. In the deeper validation-selected table, the baseline GAT reaches $42.96 \pm 0.22\%$, whereas the selected GraphSAGE+PairNorm configuration reaches $48.00 \pm 0.61\%$. This corresponds to a difference of approximately

5.04 percentage points between the selected configurations. In a separate matched GraphSAGE comparison at eight layers and 128 hidden dimensions, the archived runs yield approximately 36.79% without PairNorm and 48.00% with PairNorm, a difference of about 11.21 percentage points. However, the average matched difference across all evaluated configurations on Amazon-Ratings in the deeper phase is approximately 3.09 percentage points. This illustrates that the effect of PairNorm can vary considerably across model configurations.

The screening also contains cases in which adding PairNorm candidates provides little or no improvement. On Cora and CiteSeer, the selected configurations in each phase are unchanged between the baseline and full candidate sets. On Squirrel, the deeper full selection reaches $31.76 \pm 0.85\%$, compared with $31.99 \pm 1.65\%$ for the deeper baseline. The two selected configurations also differ in depth and hidden dimension. This result does not support a general claim that PairNorm improves deep-GNN performance.

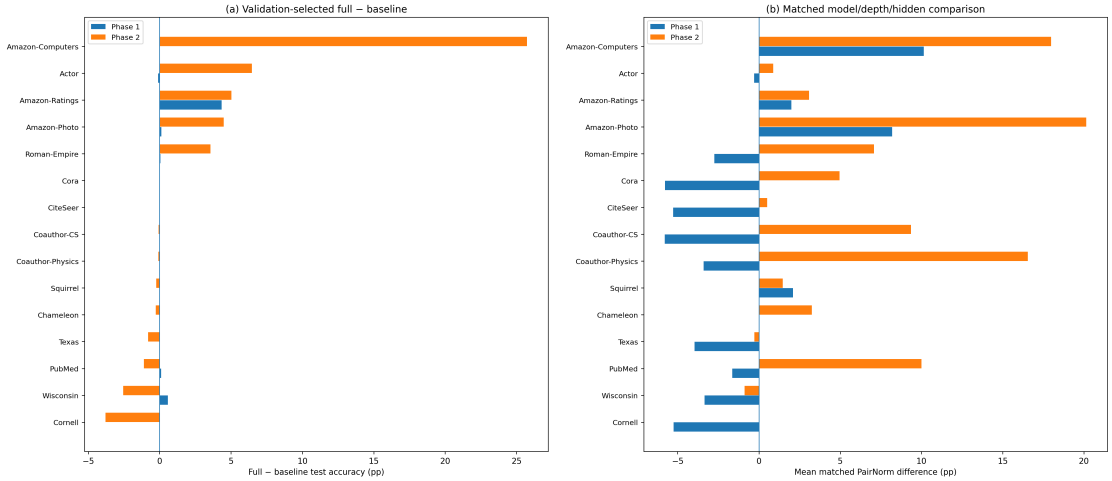


Figure 7: PairNorm comparisons in the broad real-world screening. The left panel compares the test accuracy of validation-selected baseline and full configurations, while the right panel summarizes accuracy differences between models with and without PairNorm under matched architecture, depth, and hidden dimension.

Taken together, the broad screening provides an initial answer to RQ1: depth-related predictive degradation is strongly dependent on the dataset and selected model configuration. Some deeper models lose a substantial amount of accuracy, while others remain competitive with their shallow counterparts. PairNorm can improve performance in certain settings, but the effect is neither uniform nor sufficient to establish an oversmoothing explanation.

This limitation motivates the following study. Instead of continuing to compare the best configurations from heterogeneous model families, the analysis narrows its focus to matched GraphSAGE and GraphSAGE+PairNorm experiments on four deliberately contrasting datasets.

Table 2: Validation-selected results from the broad real-world screening. Phase 1 considers depths 2 and 4. Phase 2 considers depths 8, 16, and 32. Each entry identifies the selected model, depth, hidden dimension, and test accuracy. The baseline and full searches were conducted using separate training runs. Consequently, identical model configurations may have different reported test accuracies. These differences should not be interpreted as isolated effects of PN, where PN denotes PairNorm.

Dataset	Phase 1 baseline	Phase 1 full	Phase 2 baseline	Phase 2 full
Cora	GAT L4, H128 79.50%	GAT L4, H128 79.50%	GCN L8, H64 $76.70 \pm 0.85\%$	GCN L8, H64 $76.70 \pm 0.85\%$
CiteSeer	GraphSAGE L2, H128 70.60%	GraphSAGE L2, H128 70.60%	GCN L8, H128 $63.27 \pm 0.31\%$	GCN L8, H128 $63.27 \pm 0.31\%$
PubMed	GraphSAGE L2, H64 77.20%	GAT+PN L4, H128 77.30%	GAT L8, H128 $77.57 \pm 1.38\%$	GAT+PN L8, H128 $76.47 \pm 0.95\%$
Roman-Empire	GraphSAGE L4, H128 $79.37 \pm 0.58\%$	GraphSAGE L4, H128 $79.42 \pm 0.51\%$	GraphSAGE L8, H128 $68.06 \pm 0.67\%$	GraphSAGE+PN L8, H128 $71.62 \pm 0.78\%$
Amazon-Photo	GAT L2, H128 $91.51 \pm 0.57\%$	GAT L2, H128 $91.63 \pm 0.60\%$	GCN L8, H64 $82.58 \pm 2.57\%$	GraphSAGE+PN L8, H128 $87.07 \pm 1.30\%$
Amazon-Computers	GCN L2, H128 $83.71 \pm 0.79\%$	GCN L2, H128 $83.74 \pm 0.58\%$	GCN L8, H64 $56.63 \pm 9.74\%$	GraphSAGE+PN L8, H128 $82.37 \pm 1.47\%$
Coauthor-CS	GCN L2, H128 $91.22 \pm 0.69\%$	GCN L2, H128 $91.22 \pm 0.69\%$	GCN L8, H64 $86.17 \pm 0.92\%$	GCN L8, H64 $86.09 \pm 0.90\%$
Coauthor-Physics	GCN L2, H64 $93.20 \pm 0.54\%$	GCN L2, H64 $93.20 \pm 0.54\%$	GCN L8, H64 $92.30 \pm 0.56\%$	GCN L8, H64 $92.21 \pm 0.49\%$
Actor	GraphSAGE L4, H64 $34.49 \pm 1.11\%$	GraphSAGE L4, H64 $34.39 \pm 1.15\%$	GCN L8, H64 $26.40 \pm 1.25\%$	GraphSAGE+PN L8, H128 $32.87 \pm 0.60\%$
Chameleon	GraphSAGE L2, H64 $49.82 \pm 2.32\%$	GraphSAGE L2, H64 $49.82 \pm 2.32\%$	GraphSAGE L8, H64 $42.83 \pm 2.80\%$	GraphSAGE L8, H64 $42.57 \pm 2.74\%$
Squirrel	GraphSAGE L2, H64 $36.95 \pm 2.27\%$	GraphSAGE L2, H64 $36.95 \pm 2.27\%$	GraphSAGE L8, H128 $31.99 \pm 1.65\%$	GraphSAGE+PN L16, H64 $31.76 \pm 0.85\%$
Cornell	GraphSAGE L2, H128 $71.08 \pm 5.56\%$	GraphSAGE L2, H128 $71.08 \pm 5.56\%$	GraphSAGE L8, H64 $56.22 \pm 6.96\%$	GraphSAGE+PN L8, H64 $52.43 \pm 12.30\%$
Texas	GraphSAGE L2, H128 $78.65 \pm 5.76\%$	GraphSAGE L2, H128 $78.65 \pm 5.76\%$	GraphSAGE L8, H128 $65.68 \pm 7.21\%$	GraphSAGE L8, H128 $64.86 \pm 7.32\%$
Wisconsin	GraphSAGE L2, H128 $80.20 \pm 4.66\%$	GraphSAGE L2, H128 $80.78 \pm 4.51\%$	GraphSAGE L8, H128 $66.67 \pm 7.22\%$	GraphSAGE L8, H128 $64.12 \pm 7.28\%$
Amazon-Ratings	GraphSAGE L4, H128 $45.76 \pm 0.41\%$	GraphSAGE+PN L4, H128 $50.10 \pm 0.41\%$	GAT L8, H128 $42.96 \pm 0.22\%$	GraphSAGE+PN L8, H128 $48.00 \pm 0.61\%$

5 Selected Real-World Cases

The broad screening in the previous section showed that depth-related degradation and PairNorm effects vary substantially across datasets. However, the validation-selected comparisons do not necessarily isolate the effect of normalization, as the selected configurations may differ in architecture and hyperparameters. The next experiment therefore focuses on matched GraphSAGE comparisons across four datasets and examines both predictive performance and hidden-representation geometry.

The four selected datasets are Amazon-Ratings, Squirrel, Roman-Empire, and Actor. They were chosen because the preceding experiments indicated contrasting relationships between depth and normalization, including cases in which PairNorm appeared helpful and cases in which it did not. The datasets are therefore selected to explore different underlying mechanisms, rather than to form a random or representative sample of all possible graph-learning tasks.

For each dataset, GraphSAGE and GraphSAGE+PairNorm are compared at depths $L = 4$ and $L = 8$, with a hidden dimension of 128. The design uses ten matched splits and a training budget of 800 epochs. The learning-rate candidates are 0.0003, 0.001, 0.003, 0.005, 0.01 and 0.03. The matching allows depth and normalization comparisons to be evaluated under the same learning rate and corresponding split.

Table 3: Design of the four selected real-world comparisons. All datasets use the same architecture pair, depth values, hidden dimension, number of matched splits, and training budget.

Dataset	Models	Depths	Hidden	Splits
Amazon-Ratings	GS / GS+PN	4, 8	128	10
Squirrel	GS / GS+PN	4, 8	128	10
Roman-Empire	GS / GS+PN	4, 8	128	10
Actor	GS / GS+PN	4, 8	128	10

Note. GS = GraphSAGE; PN = PairNorm.

5.1 Depth Interaction

A comparison of the eight-layer models alone would show whether PairNorm changes performance at a fixed depth. It would not, however, establish whether PairNorm specifically reduces the degradation associated with increasing depth. For this reason, the analysis compares the change from four to eight layers for both models.

Let Acc_{L4} and Acc_{L8} denote test accuracy at the validation-selected checkpoint for models with four and eight message-passing layers, respectively. Define the depth loss as

$$\mathrm{DepthLoss} = \mathrm{Acc}_{L4} - \mathrm{Acc}_{L8}\,. \tag{16}$$

A positive depth loss indicates that the deeper network performs worse, whereas a negative value would mean that the eight-layer model achieves higher accuracy.

The effect of PairNorm on this relationship is evaluated using a matched difference-in-differences (DiD) comparison, defined as follows:

$$\begin{aligned}\mathrm{DiD} = (&\mathrm{Acc}_{L8}^{PN} - \mathrm{Acc}_{L4}^{PN}) \\ &- (\mathrm{Acc}_{L8}^{GS} - \mathrm{Acc}_{L4}^{GS}).\end{aligned} \tag{17}$$

Here, GS denotes GraphSAGE without PairNorm, while PN in this case denotes GraphSAGE+PairNorm. In other words, the interaction measures the depth-related performance loss of GraphSAGE minus that of GraphSAGE+PairNorm. Positive values indicate that PairNorm reduces the loss associated with increasing depth, whereas negative values indicate that the depth-related degradation becomes larger under PairNorm.

Figure 8 shows that the sign of this interaction changes across the four datasets. Amazon-Ratings and Squirrel exhibit positive interactions, while Roman-Empire and Actor exhibit negative interactions.

For Amazon-Ratings, GraphSAGE loses 2.734 percentage points when depth increases from four to eight layers. The corresponding loss under PairNorm is 0.928 percentage points. Thus, PairNorm reduces depth-related degradation by 1.806 percentage points in the matched comparison. The difference is meaningful because the intervention reduces the depth penalty rather than merely changing the selected performance of one isolated deep model.

Squirrel shows a larger favorable interaction. The GraphSAGE depth loss is 4.092 percentage points, while GraphSAGE+PairNorm loses only 0.893 percentage points. This corresponds to a positive interaction of 3.199 percentage points. In this configuration, PairNorm helps preserve a greater proportion of the predictive performance achieved by the four-layer model as network depth increases.

The opposite occurs on Roman-Empire. The GraphSAGE depth loss is only 0.184 percentage points, whereas the PairNorm variant loses 2.243 percentage points. The negative interaction of approximately −2.060 percentage points suggests that PairNorm does not consistently mitigate depth-related performance degradation. In this particular matched comparison, its inclusion is associated with a larger accuracy loss as network depth increases.

Actor provides another negative interaction. GraphSAGE loses 1.178 percentage points from four to eight layers, while GraphSAGE+PairNorm loses 3.132 percentage points. The resulting interaction is −1.954 percentage points. Thus, although PairNorm changes the hidden representations, it again does not reduce depth-related degradation in this condition.

The matched interaction results are summarized in Table 4. In addition to the interaction values, the table reports Holm-adjusted p-values and Cohen's d_z effect sizes to assess the statistical evidence and the magnitude of the observed interactions.

The p-values are used to test whether the mean depth interaction differs from zero. Since this test is performed separately for four datasets, the probability of obtaining at least one false-positive result increases. Holm's sequential multiple-testing correction addresses this problem by controlling the family-wise error rate across the four comparisons [Hol79]. A small adjusted p-value therefore provides evidence that the interaction differs from zero, even after accounting for multiple testing.

However, statistical significance alone does not indicate how large an interaction is. Cohen's d_z complements the p-values by dividing the mean paired interaction by the standard deviation of the paired interaction values [Lak13]. A larger absolute value indicates that the mean interaction is larger relative to its variability. The sign indicates its direction. Positive values correspond to a more favorable depth-related accuracy change under PairNorm, while negative values indicate the opposite.

For Amazon-Ratings, Squirrel, and Roman-Empire, the reported Holm-adjusted p-value is approximately 0.005859. For Actor, the corresponding value is 0.011719. The observed interactions are therefore distinguishable from zero under the reported paired testing procedure. However, these results do not estab-

lish that PairNorm has a consistent effect across datasets. The four datasets were deliberately selected based on interesting behavior observed in earlier experiments, rather than being randomly sampled from a broader set of graph-learning tasks. Therefore, the findings should be interpreted as exploratory evidence for the investigated configurations, without assuming that the same effects generalize to other datasets.

The paired effect sizes further highlight the dataset-dependent differences in the depth interaction. Amazon-Ratings and Squirrel have positive d_z values of approximately 2.316 and 2.977, respectively, indicating favorable interactions relative to the variability of the paired differences. Roman-Empire and Actor instead have negative values of approximately −2.001 and −1.291, indicating unfavorable interactions. These results show that both the direction and the standardized magnitude of the depth interaction vary across datasets.

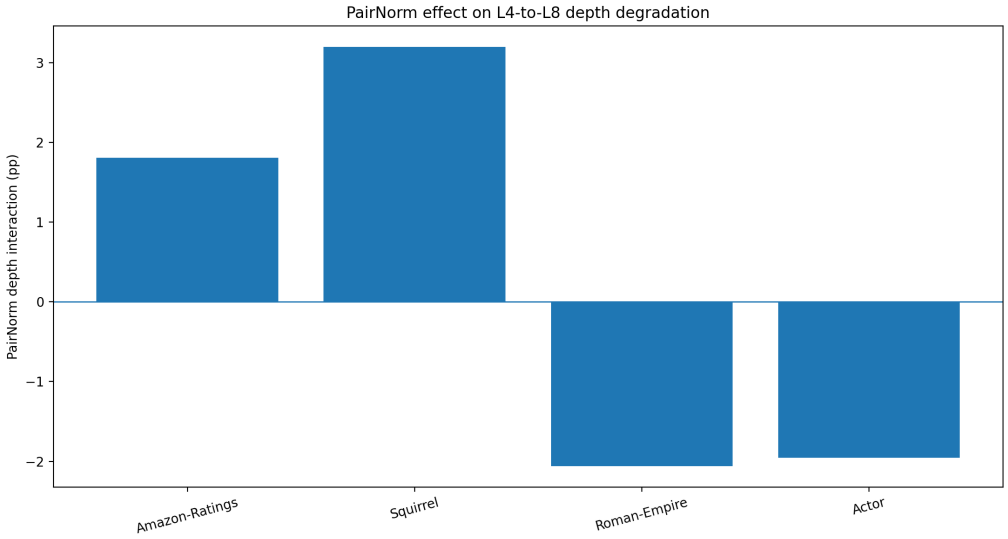


Figure 8: Matched depth interaction for GraphSAGE and GraphSAGE+PairNorm on four real-world datasets. Positive values indicate that PairNorm reduces the test-accuracy loss from four to eight layers; negative values indicate increased degradation under PairNorm. All differences are expressed in percentage points.

Table 4: Matched depth interactions and representation-diagnostic ratios for the four selected real-world datasets. Depth losses and DiD are expressed in percentage points. NDE and rank ratios compare PairNorm with GraphSAGE at eight layers.

Dataset	LR	GS loss	PN loss	DiD	Holm p	d_z	NDE ratio	Rank ratio
Amazon-Ratings	0.003	2.734	0.928	+1.806	0.005859	+2.316	2.31	3.63
Squirrel	0.030	4.092	0.893	+3.199	0.005859	+2.977	3.14	2.30
Roman-Empire	0.005	0.184	2.243	-2.060	0.005859	-2.001	1.61	0.99
Actor	0.0003	1.178	3.132	-1.954	0.011719	-1.291	1.47	2.30

Note. LR = learning rate; GS = GraphSAGE; PN = GraphSAGE+PairNorm; DiD = difference-in-differences; d_z = paired standardized effect size. Representation ratios refer to epoch 800.

5.2 Representation Diagnostics

The preceding results establish that PairNorm changes depth-related performance differently across the four datasets. The next question is whether these differences correspond to changes in the geometry of the node representations.

Figure 9 compares the normalized Dirichlet energy and effective-rank ratio of eight-layer GraphSAGE models with and without PairNorm. Both measurements are evaluated at epoch 800 under the matched learning-rate settings. Each bar represents the ratio of the PairNorm value to the corresponding GraphSAGE value. A ratio greater than one therefore indicates a larger diagnostic value under PairNorm.

On Amazon-Ratings, PairNorm increases NDE by a factor of approximately 2.31 and the effective-rank ratio by a factor of approximately 3.63. These changes in representation geometry are observed alongside the positive depth interaction observed in the predictive results. Together, the findings suggest that PairNorm helps maintain representation diversity in a setting where deeper GraphSAGE models otherwise exhibit greater accuracy degradation.

Squirrel displays a similar relationship. NDE increases by a factor of approximately 3.14, while the effective-rank ratio increases by a factor of approximately 2.30. At the same time, PairNorm reduces the depth loss by 3.199 percentage points. Thus, in both datasets where PairNorm is associated with a favorable depth interactions, the node representations also show greater diversity.

However, this relationship does not generalize to all four datasets. Roman-Empire shows an NDE ratio of approximately 1.61 but an effective-rank ratio of approximately 0.99. While local representation variation increases under PairNorm, effective dimensionality remains almost unchanged. The predictive depth interaction is negative. This case demonstrates that a larger Dirichlet-energy diagnostic does not necessarily correspond to stronger predictive performance as mentioned before.

Actor provides an especially informative counterexample. PairNorm increases NDE by a factor of approximately 1.47 and the effective-rank ratio by a factor of approximately 2.30. Nevertheless, the depth interaction is -1.954 percentage points. In other words, the eight-layer PairNorm model maintains greater measured representation diversity while suffering a larger relative decline in predictive accuracy.

This counterexample is important for interpreting oversmoothing. Increasing representation diversity can change the geometry of hidden vectors without necessarily improving their usefulness for node classification. A representation may remain dispersed or occupy several effective dimensions while failing to encode information that separates the relevant classes.

The selected-four results therefore support two conclusions. First, PairNorm can reduce depth-related degradation in some real-world settings, and the favorable cases can coincide with increased representation diversity. Second, maintaining greater NDE or effective rank is not sufficient to ensure stronger predictive performance. The evidence is consistent with oversmoothing being relevant in certain configurations, but it also indicates that the relationship between geometry and classification depends on additional dataset and task properties.

Because the four datasets differ in many characteristics beyond homophily, these comparisons cannot identify which graph properties are responsible for the observed variation. The following controlled experiments address this limitation by changing the relationship between graph edges and labels within the same underlying dataset.

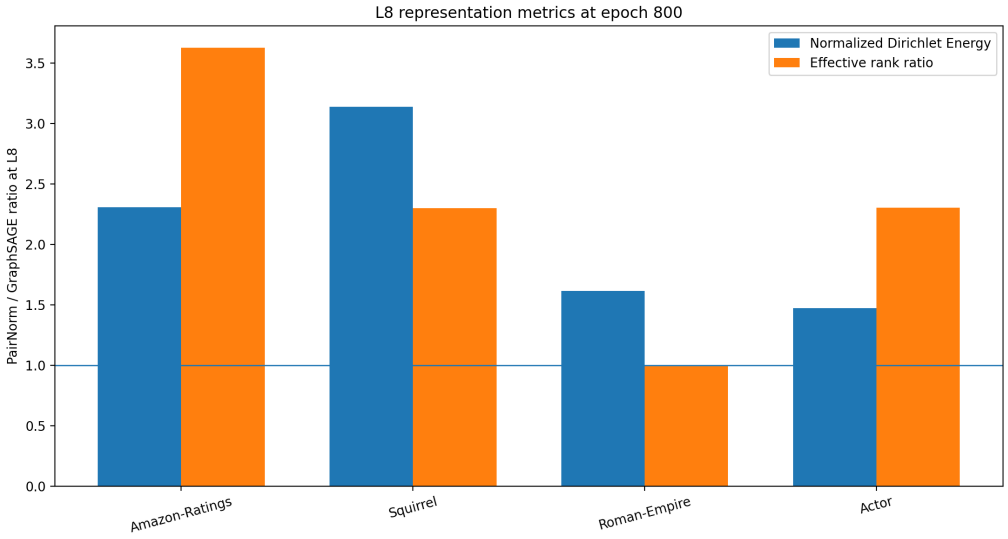


Figure 9: PairNorm-to-GraphSAGE ratios of normalized Dirichlet energy and effective-rank ratio at eight layers and epoch 800. Values greater than one indicate increased measured representation diversity under PairNorm. The ratios do not imply a corresponding improvement in predictive accuracy.

6 Controlled Homophily

The selected real-world experiments demonstrated that the effect of PairNorm differs across datasets, even when architecture, depth, and hidden dimension are matched. However, comparing different datasets does not reveal whether these differences arise from homophily, node features, class distributions, or other structural properties. The controlled-homophily experiment addresses this limitation by modifying graph connectivity within a fixed real-world node-classification setting.

6.1 Controlled Graph Construction

The experiment uses PubMed and Roman-Empire as two contrasting graph-learning environments. PubMed has relatively high raw edge homophily in its natural topology, while Roman-Empire is strongly heterophilous. Both datasets retain their original node features and class labels, but their graph edges are modified to obtain specified target homophily values.

The controlled graphs target raw edge homophily values of

$$h \in \{0.1, 0.5, 0.9\}. \tag{18}$$

A value of $h = 0.1$ indicates that approximately 10% of the undirected edges connect nodes with the same class label. At $h = 0.9$, approximately 90% connect nodes belonging to the same class. The intermediate setting $h = 0.5$ provides a third condition between these two extremes.

The graph-construction procedure uses degree-preserving double-edge swaps. A swap replaces two existing edges with two alternative connections between their endpoints, subject to validity constraints. This

type of operation can change the relationships between neighboring nodes without changing the number of incident edges at each involved node. Degree-preserving graph randomization is a known structural-control principle [MS02], while the specific homophily-targeting procedure used here is an experimental construction for this thesis.

The implemented procedure restricts swaps so that the graph remains a simple undirected graph and rejects changes that would violate the required connectivity condition. As a result, the experiments preserve the number of nodes, number of edges, and degree sequence while modifying the alignment of edges with class labels. Node features and labels remain unchanged.

These constraints are useful because the experiment does not simply compare unrelated datasets with different feature representations and class distributions. Instead, it changes a defined structural characteristic while holding several other quantities fixed. Nevertheless, degree-preserving rewiring also changes higher-order structural properties such as neighborhood composition, paths, and local connectivity patterns. The experiment therefore identifies the effect of the controlled graph intervention, not a mathematically isolated effect of a scalar homophily value independent of all topology.

For the low-label experiments, 5% of nodes are assigned to the training set, 10% to validation, and 85% to the test set. This setting is intentionally demanding because the models must learn from a relatively small number of labeled nodes.

The initial controlled exploration evaluates several network depths, including two, four, and eight message-passing layers. A subsequent factorial study concentrates on seven and eight layers to investigate the behavior near the observed deep-network degradation regimes more carefully.

The refined factorial design uses GraphSAGE and GraphSAGE+PairNorm, 128 hidden dimensions, five graph seeds, and five initialization seeds. Each configuration is trained for 2400 epochs, and the evaluated mask is split 0.

For two datasets, three homophily targets, two model variants, two depths, five graph seeds, and five initialization seeds, the design contains

$$2 \cdot 3 \cdot 2 \cdot 2 \cdot 5 \cdot 5 = 600 \tag{19}$$

experimental cells.

Table 5: Refined controlled-homophily factorial design. The experiment varies graph topology, depth, and PairNorm while retaining the underlying node features and labels within each dataset.

Factor	Setting
Datasets	PubMed-derived; Roman-Empire-derived
Target raw edge homophily	0.1, 0.5, 0.9
Models	GraphSAGE; GraphSAGE+PairNorm
Depths	7, 8
Hidden dimension	128
Graph seeds	1, 2, 3, 4, 5
Initialization seeds	11, 29, 47, 65, 83
Evaluated split	0
Training epochs	2400
Train / validation / test	5% / 10% / 85%
Expected / observed cells	600 / 600

6.2 Endpoint Results

The controlled experiments examine whether depth-related predictive degradation and representation collapse depend on the alignment between graph edges and class labels. The results show that changing this alignment can substantially change model behavior, but the response differs between PubMed and Roman-Empire.

Figure 10 presents the initial controlled-homophily depth exploration. It compares GraphSAGE with GraphSAGE+PairNorm across increasing depth for each dataset and homophily condition. The figure provides a broad view of how depth and normalization interact before the more focused factorial analysis.

In PubMed, the low-homophily conditions show a pronounced separation between the two models at greater depth. GraphSAGE loses much of its predictive performance, while the PairNorm variant retains substantially higher accuracy. The difference is especially visible at eight layers. In the high-homophily condition, both models remain more competitive, and the contrast is considerably smaller.

Roman-Empire exhibits a different pattern. Its response varies with homophily, and PairNorm does not provide the same consistent recovery seen in the difficult PubMed conditions. At high homophily, the baseline can perform very strongly without normalization. This behavior illustrates that PairNorm's benefit cannot be predicted from increasing depth or decreasing homophily alone.

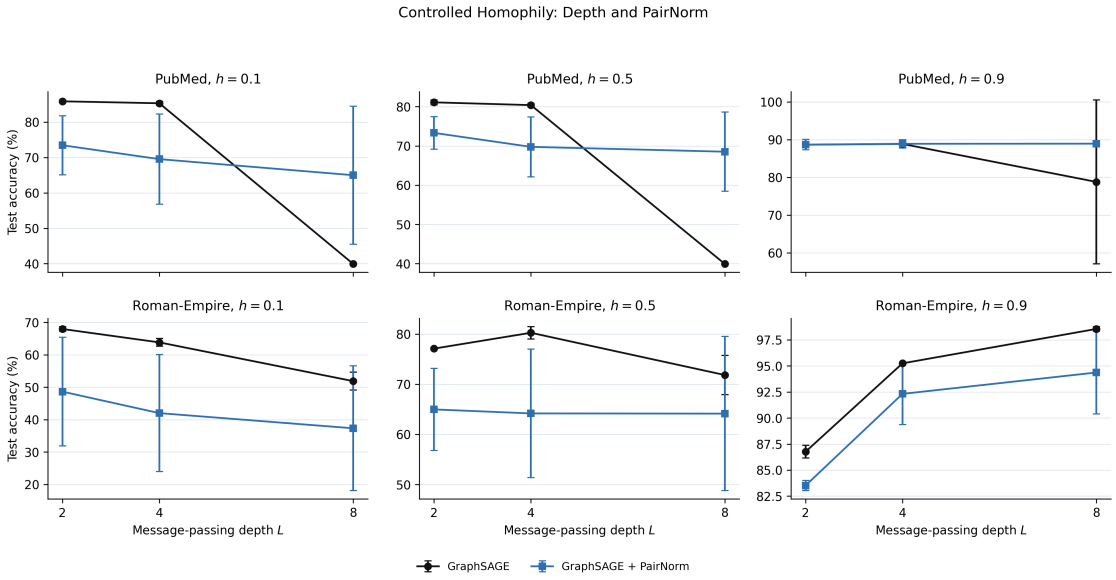


Figure 10: Initial controlled-homophily depth exploration for PubMed-derived and Roman-Empire-derived graphs. The panels compare GraphSAGE with and without PairNorm at target raw edge-homophily levels of 0.1, 0.5, and 0.9. Predictive accuracy is measured at validation-selected checkpoints.

The refined eight-layer endpoint results provide a more detailed comparison and are summarized in Table 6. The table reports test accuracy at the validation-selected checkpoint together with normalized Dirichlet energy and effective rank measured at epoch 2400. These two measurement times must be distinguished when interpreting the results.

For PubMed at $h = 0.1$, GraphSAGE reaches only 39.94% test accuracy, whereas GraphSAGE+PairNorm reaches 83.49%. The improvement is 43.55 percentage points. This is one of the largest predictive differences observed in the real-world-derived oversmoothing experiments.

The representation diagnostics reveal an equally pronounced contrast. GraphSAGE has normalized Dirichlet energy of approximately 5.20×10^{-26} and an effective rank of 1. These results suggest that the hidden representations are highly compressed. Neighboring nodes have almost no measured squared representation difference relative to their overall magnitude, while most of the variation is concentrated along a single effective direction.

Under PairNorm, NDE rises to approximately 2.652 and effective rank to 4.688. Thus, the normalized model retains substantially greater edgewise variation and effective dimensionality. The substantial decline in predictive performance, together with strongly compressed node representations and their improvement under PairNorm, provides strong evidence consistent with an oversmoothing-related failure in this specific controlled condition.

A similar outcome appears for PubMed at $h = 0.5$. GraphSAGE again reaches 39.94% accuracy, while GraphSAGE+PairNorm reaches 78.09%, an improvement of 38.15 percentage points. GraphSAGE NDE is approximately 2.79×10^{-20} and its effective rank is 1. PairNorm increases the respective values to 1.404 and 4.814. The identical baseline accuracies at $h = 0.1$ and $h = 0.5$, together with the almost collapsed node representations, suggest that the tested deep GraphSAGE configuration exhibits severe degradation under both homophily conditions.

At $h = 0.9$, PubMed behaves differently. Baseline GraphSAGE reaches 78.80%, and the PairNorm variant reaches 88.56%, corresponding to a smaller improvement of 9.76 percentage points. The baseline NDE of 0.134 and effective rank of 2.534 are no longer as close to complete collapse as in the two lower-homophily settings. PairNorm increases these measurements to 0.231 and 3.476. This contrast shows that the same dataset and architecture can exhibit substantially different representation and performance behavior after a controlled topology modification.

The Roman-Empire results demonstrate that the phenomenon is not determined by the homophily target alone. At $h = 0.1$, GraphSAGE reaches 51.89% and PairNorm reaches 55.56%, a modest improvement of 3.67 percentage points. Both models exhibit substantial NDE values of 1.211 and 1.685, respectively, along with effective ranks of 11.278 and 16.968. In contrast to the corresponding PubMed setting, the Roman-Empire baseline does not exhibit the same near-zero energy and rank-one representation collapse.

At $h = 0.5$, Roman-Empire GraphSAGE reaches 71.82%, compared with 78.68% under PairNorm. The improvement of 6.86 percentage points accompanies an NDE increase from 0.284 to 0.726 and an effective-rank increase from 11.579 to 20.027. This indicates a favorable association between preserved representation diversity and predictive performance, although the relationship is much less extreme than in low-homophily PubMed.

The clearest counterexample appears at Roman-Empire $h = 0.9$. Baseline GraphSAGE reaches 98.55% accuracy, while GraphSAGE+PairNorm reaches 98.13%. PairNorm therefore produces a small negative difference of 0.42 percentage points. Nevertheless, NDE increases from 0.082 to 0.187 and effective rank increases from 9.969 to 17.178. Normalization leads to substantial changes in representation geometry, but these changes are not accompanied by an improvement in predictive performance.

This result demonstrates that greater representation diversity is not sufficient for improved classification. When the baseline already performs very strongly, additional geometric spread may not provide useful information for the task. Normalization may also modify node representations that already contain information effectively used by the classifier. Although the experiment does not determine which explanation plays the greater role, the results indicate that greater representation diversity does not necessarily lead to higher predictive accuracy.

Table 6: Eight-layer endpoint results in the refined low-label controlled-homophily experiment. Accuracy is evaluated at the best-validation checkpoint. NDE and effective rank are measured at epoch 2400.

Dataset	h	GS acc.	PN acc.	PN effect	GS NDE	PN NDE	GS rank	PN rank
PubMed	0.1	39.94%	83.49%	+43.55 pp	5.20×10^{-26}	2.652	1.000	4.688
PubMed	0.5	39.94%	78.09%	+38.15 pp	2.79×10^{-20}	1.404	1.000	4.814
PubMed	0.9	78.80%	88.56%	+9.76 pp	0.134	0.231	2.534	3.476
Roman-Empire	0.1	51.89%	55.56%	+3.67 pp	1.211	1.685	11.278	16.968
Roman-Empire	0.5	71.82%	78.68%	+6.86 pp	0.284	0.726	11.579	20.027
Roman-Empire	0.9	98.55%	98.13%	-0.42 pp	0.082	0.187	9.969	17.178

Note. GS = GraphSAGE; PN = GraphSAGE+PairNorm; acc. = test accuracy; pp = percentage points; h = target raw edge homophily. PN effect is PN accuracy minus GS accuracy.

The comparison between PubMed and Roman-Empire is important for the interpretation of homophily. Both datasets were rewired toward the same numerical targets, yet their predictive behavior and representation diagnostics differed substantially. Moreover, because the datasets have different numbers of classes and class distributions, the same raw edge-homophily value does not necessarily indicate the same level of same-class connectivity relative to what would be expected by chance. Therefore, identical homophily targets do not necessarily imply the same tendency for nodes to connect to others with the same label across the two datasets.

The controlled experiments suggest that the underlying mechanisms depend on the experimental conditions. In the challenging PubMed settings, substantial representation compression occurs alongside a decline in predictive performance, while PairNorm is associated with improvements in both. However, the Roman-Empire counterexample shows that substantial geometric changes do not necessarily yield higher accuracy. Homophily therefore influences the setting in which a failure occurs, but it is not a sufficient explanation by itself.

6.3 Temporal Representation Dynamics

Endpoint diagnostics indicate the final state of the learned representations, but they do not show how the models reach that state. In particular, a near-zero NDE at the final epoch could reflect persistent compression throughout training or a collapse that develops only after an earlier period of useful representation learning. These possibilities have different implications for interpreting the relationship between optimization and oversmoothing.

Figure 11 therefore tracks NDE and the effective-rank ratio over the training epochs for two contrasting controlled conditions: PubMed at $h = 0.1$ and Roman-Empire at $h = 0.9$, both with eight message-passing layers.

For PubMed at $h = 0.1$, the GraphSAGE baseline remains close to zero in NDE over most of the recorded training trajectory. Its effective-rank ratio is likewise concentrated near the low end of the scale. By contrast, GraphSAGE+PairNorm maintains substantially greater NDE throughout training and preserves a larger effective-rank ratio. The distinction between the models is therefore not limited to their final recorded values.

The consistent difference suggests that the deep PubMed baseline maintains highly compressed node representations. However, the curves do not show whether this compression is caused by the training process itself. Node representations may already become very similar early on, since repeated message passing can reduce their diversity even before the model parameters have been extensively optimized. The results therefore show that the compression occurs and persists, but they do not identify its precise cause.

The Roman-Empire condition exhibits a different temporal pattern. Both models retain nonzero representation diversity, and PairNorm generally increases the measured NDE and effective-rank ratio. However, the baseline already reaches very high predictive accuracy, and the PairNorm model does not improve it. The temporal measurements therefore reinforce the endpoint counterexample: the geometry can change consistently throughout training without producing a useful gain in node classification.

Taken together, the controlled-homophily and temporal analyses extend the endpoint findings and reinforce the conclusion that representation geometry alone cannot fully explain predictive performance. This motivates a further controlled investigation in which graph structure and node-feature information can be varied independently. A synthetic contextual stochastic block model is used for this purpose.

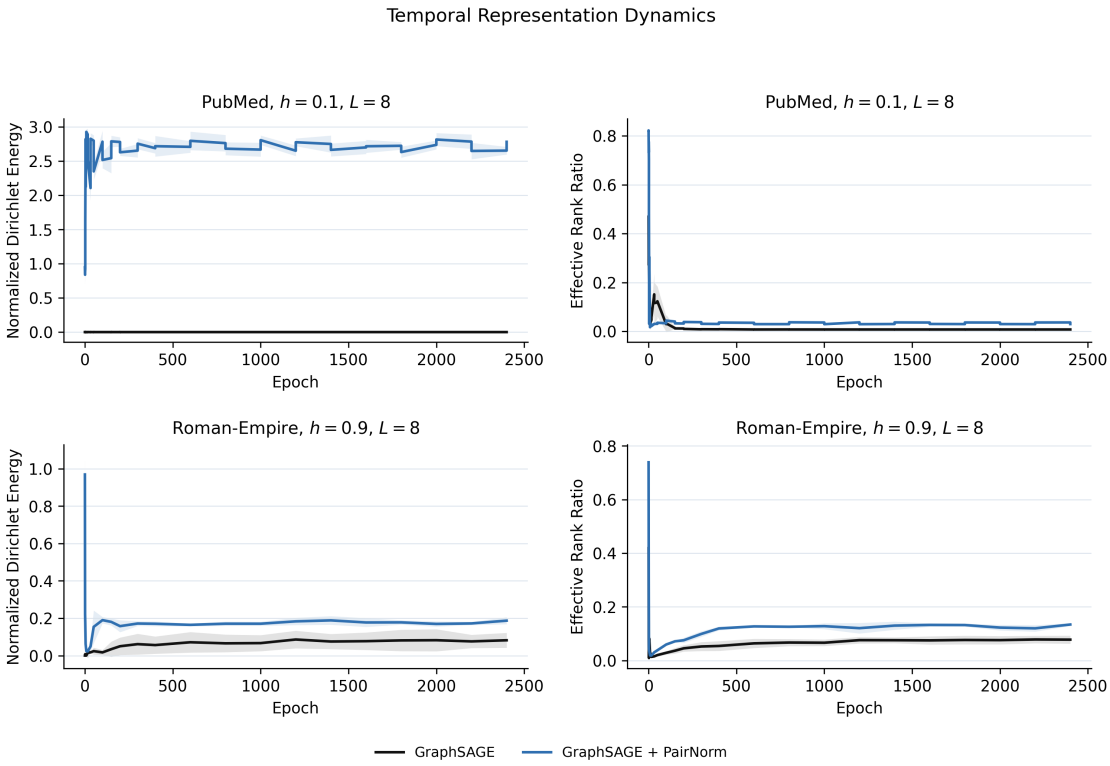


Figure 11: Temporal representation dynamics for controlled PubMed at $h = 0.1$ and Roman-Empire at $h = 0.9$, both at eight layers. The curves show normalized Dirichlet energy and effective-rank ratio across training epochs. Persistent representation compression and non-collapsed dynamics are compared without assuming that training itself is the sole cause of the observed geometry.

7 Controlled cSBM Experiments

The controlled real-world experiments retain the node features and labels of PubMed and Roman-Empire while modifying graph connectivity. This design reduces several sources of variation but remains dependent on the properties of the original datasets. In particular, the class distribution and the relationship between node features and labels are not independently adjustable.

To investigate whether the observed behavior extends to a setting with more explicit control over these quantities, the next experiment uses a synthetic graph-learning task based on a contextual stochastic block model (cSBM).

7.1 Synthetic Graph and Feature Construction

A stochastic block model generates graph connections according to a partition of the nodes into groups or communities. In a contextual stochastic block model, nodes additionally possess feature vectors related to the same underlying group structure. The combination permits experiments in which both connectivity and node attributes provide information about latent classes.

Deshpande et al. introduce and analyze contextual stochastic block models that combine a graph with high-dimensional node covariates associated with latent communities [DSMM18]. Their work provides the conceptual basis for the synthetic setting used here. However, the graph generator in this thesis is a controlled experimental variant and does not reproduce the original probabilistic model of Deshpande et al. exactly. The specific node count, edge-generation parameters, Gaussian feature construction, and experimental splits are choices of this study.

Each generated graph contains 2,000 nodes belonging to two balanced classes of 1,000 nodes each. Every node receives a 64-dimensional feature vector. The target average degree is 20, and the class-label edge-homophily targets are again

$$h \in \{0.1, 0.5, 0.9\}. \tag{20}$$

The features are generated from isotropic Gaussian noise together with a class-dependent shift along a randomly chosen unit direction. The separation between the class means is controlled by the feature-signal parameter,

$$\mathrm{FS} \in \{0.25, 0.50\}. \tag{21}$$

The feature-signal parameter therefore controls the magnitude of the class-related component in the generated features. Increasing it strengthens the class-mean separation under this construction, without changing the target graph homophily.

Because the classes are balanced, the expected same-class edge fraction under class-independent random mixing is approximately 0.5. Consequently, the intermediate homophily target of 0.5 corresponds to an approximately class-neutral connectivity regime, whereas $h = 0.1$ represents a strongly heterophilous relationship and $h = 0.9$ a strongly homophilous one. Both extremes can contain substantial class information in the connectivity pattern, although the direction of the relationship differs.

This distinction matters for the interpretation of the synthetic results. Low homophily is not equivalent to a lack of useful graph information. In a balanced two-class graph, a strong tendency to connect nodes from the opposite class can itself be informative for classification. The usefulness of that pattern depends

on whether the architecture and optimization procedure can exploit it.

The experiment uses 5% training nodes, 10% validation nodes, and 85% test nodes. For the 2,000-node balanced graph, this corresponds to 100 training nodes, 200 validation nodes, and 1,700 test nodes. The splits are stratified so that the class proportions remain balanced. Ten possible stratified masks are generated, of which split 0 is evaluated.

The compared models are GraphSAGE and GraphSAGE+PairNorm, using a hidden dimension of 128. The principal controlled design evaluates depths of two, seven, and eight layers. Each configuration uses five coupled graph/training seeds, learning rate 0.01, weight decay 0.0005, dropout 0.5, and 200 training epochs.

The complete primary design contains 180 runs, corresponding to three homophily targets, two feature-signal strengths, two model variants, three depths, and five seeds. Additional depth investigations are used later to examine the onset of failure more closely; these follow-up runs should not be confused with the 180-run primary factorial.

Table 7: Controlled synthetic cSBM experimental design. The graph and feature generator is a thesis-specific variant inspired by the joint graph-and-covariate setting of Deshpande et al. [DSMM18].

Factor	Setting
Graph family	Contextual stochastic block model variant
Nodes	2,000
Node features	64
Classes	2 balanced classes
Target edge homophily	0.1, 0.5, 0.9
Target average degree	20
Feature signal	0.25, 0.50
Feature construction	Gaussian noise + class shift
Class-mean separation	Feature-signal parameter
Models	GraphSAGE; GraphSAGE+PairNorm
Depths	2, 7, 8
Hidden dimension	128
Train / validation / test	5% / 10% / 85%
Training nodes	100
Validation nodes	200
Test nodes	1,700
Available mask splits	10 stratified splits
Evaluated split	0
Coupled seeds	1, 2, 3, 4, 5
Learning rate	0.01
Training epochs	200
Dropout	0.5
Weight decay	0.0005
Total primary runs	180

7.2 Predictive Performance Across Controlled Conditions

The synthetic experiments investigate whether the large PairNorm improvements observed on controlled PubMed can also occur in a graph where homophily and feature signal are explicitly adjustable. The

results show that they can, but that the effect depends strongly on the combination of graph structure and node-feature information.

Figure 12 compares eight-layer GraphSAGE with and without PairNorm across the three homophily levels and two feature-signal strengths. The points represent mean test accuracy over five coupled graph/training seeds, and the error bars indicate standard deviations.

At $h = 0.1$, GraphSAGE performs close to 50% accuracy for both feature-signal strengths. Because the synthetic task contains two balanced classes, this corresponds approximately to the accuracy of uninformed binary prediction. GraphSAGE+PairNorm, by contrast, reaches almost 100% accuracy in the same conditions. The result therefore reproduces a particularly large normalization-associated performance difference in a controlled synthetic setting.

This finding is noteworthy because the graph is strongly heterophilous. The result does not mean that heterophily itself causes the collapse. Rather, the connectivity contains a strong opposite-class relationship, and the PairNorm model appears able to exploit the available graph and feature information, whereas the tested deep baseline fails to do so.

At $h = 0.5$, the two models perform near 50% for the tested eight-layer configurations. PairNorm therefore does not produce the dramatic improvement observed at $h = 0.1$. This contrast is consistent with the fact that the graph connectivity is approximately class-neutral in the balanced binary setting, while the feature signal is relatively weak in the 64-dimensional Gaussian feature space. Preserving representation diversity cannot by itself create class-relevant information that is not effectively available to the predictor.

At $h = 0.9$, the result again depends on the feature signal. With feature signal 0.25, the baseline is less reliable and exhibits substantially greater variability than the PairNorm model. With feature signal 0.50, the baseline and PairNorm models both reach very high accuracy. Thus, normalization can become unnecessary for predictive recovery when the tested configuration already solves the task successfully.

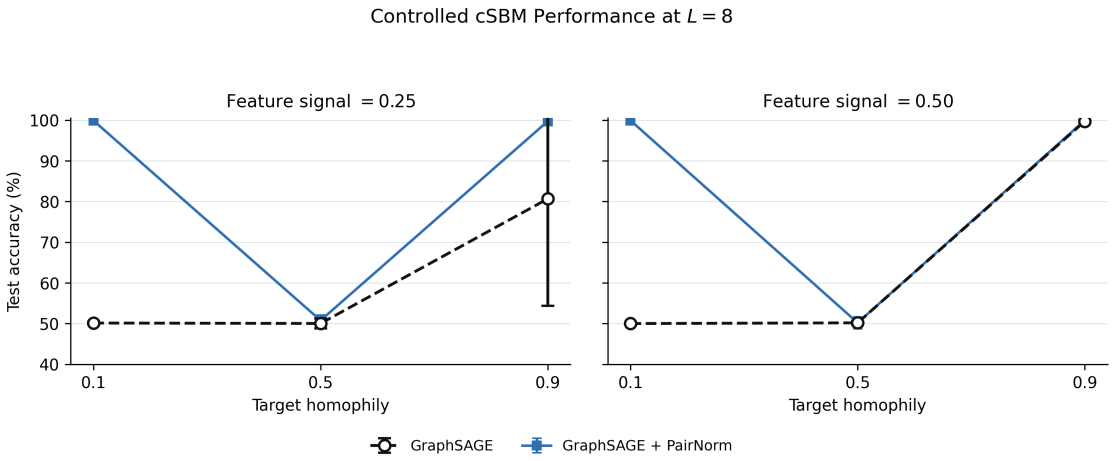


Figure 12: Test accuracy of eight-layer GraphSAGE with and without PairNorm in the controlled cSBM variants. The panels compare feature-signal strengths 0.25 and 0.50 across target homophily levels 0.1, 0.5, and 0.9. Means and standard deviations summarize five coupled graph/training seeds.

The synthetic results therefore identify two important conditions. First, PairNorm can recover almost all predictive performance in a regime where deep GraphSAGE otherwise performs near chance level. Second, PairNorm is not universally beneficial: when the available information is insufficient for the

tested model, or when the baseline already performs almost perfectly, preserving geometric spread does not necessarily improve the prediction.

7.3 Critical Depth and Representation Collapse

The large eight-layer performance contrast at $h = 0.1$ raises a more specific question: whether the baseline degrades gradually as depth increases or whether there is a sharp transition between successful and unsuccessful configurations.

Additional depth probes were therefore conducted around the critical regime. These results show a pronounced transition between seven and eight message-passing layers for the tested GraphSAGE configuration.

For feature signal 0.50 at $h = 0.1$, baseline GraphSAGE reaches approximately 99.81% at six layers and 99.71% at seven layers. Increasing depth to eight layers reduces accuracy to 50.00%. The baseline remains at approximately 50% for the additional nine- and ten-layer probes.

This sequence is important because it does not support an alternating odd-even explanation of the results. The seven-layer model already performs very strongly, whereas the eight-, nine-, and ten-layer models fail. The dominant observation is therefore a sharp failure threshold between seven and eight layers, followed by persistent degradation at the evaluated greater depths.

For feature signal 0.25, the eight-, nine-, and ten-layer baselines similarly reach approximately 50.13%, 50.00%, and 50.00%. The PairNorm variants retain approximately 99.91%, 99.64%, and 99.72%, respectively. With feature signal 0.50, the corresponding PairNorm accuracies remain near 99.9%.

Across these six deep conditions, all five baseline seeds in each condition exhibit predictive collapse, whereas none of the five corresponding PairNorm runs does. This consistency across matched repetitions indicates that the difference is not explained solely by an isolated unfavorable model initialization.

The representation diagnostics provide additional evidence. In the deep $h = 0.1$ baseline conditions, the mean pairwise cosine distance of the final hidden representations approaches zero. This means that the recorded node representations become almost aligned in direction. The corresponding PairNorm configurations retain mean pairwise cosine distances close to 0.99, indicating that a substantial degree of angular diversity remains.

These observations establish a strong association between three features of the experiment: the critical increase in depth, severe representation compression, and a collapse in predictive performance. PairNorm changes the representation geometry and prevents the predictive failure in the same controlled regime.

The evidence is therefore consistent with an oversmoothing-related mechanism contributing to the baseline failure. However, even this unusually clear intervention result should not be interpreted as proof that the measured representation collapse is the only causal pathway. PairNorm also modifies the transformations experienced by the hidden states during training, which can influence optimization and the propagation of useful signals.

A further qualification concerns the effective-rank diagnostic. The effective rank does not follow every accuracy difference monotonically across the synthetic conditions. In particular, preserving a larger rank or pairwise distance is not always associated with stronger prediction. This reinforces the need to interpret multiple diagnostics together rather than treating one statistic as a universal indicator of successful information preservation.

The cSBM results therefore provide one of the strongest controlled examples supporting an oversmoothing interpretation in this thesis, while also demonstrating its limitations. In the difficult heterophilous

deep-network regime, predictive and geometric collapse occur together and PairNorm prevents both. In other graph-feature conditions, the same normalization does not necessarily produce a meaningful predictive advantage.

The remaining experiments examine whether these patterns are robust to changes in optimization settings and whether the different operations of PairNorm contribute equally to the observed outcomes.

8 Learning-Rate Sensitivity

The preceding controlled experiments identified configurations in which deeper GraphSAGE models exhibit large predictive losses accompanied by strongly compressed representations. However, an observed failure at a particular training configuration does not establish that the architecture cannot learn the task. Optimization settings may influence whether the model reaches a useful solution, and PairNorm may change these optimization dynamics in addition to modifying representation geometry.

The learning-rate sensitivity experiment investigates this competing explanation directly. It focuses on the controlled $h = 0.1$ versions of PubMed and Roman-Empire, using eight-layer GraphSAGE with and without PairNorm and a hidden dimension of 128.

The learning rate determines the step size used when updating the model parameters during optimization. If the learning rate is too small for the available training budget, optimization may progress too slowly. Conversely, an excessively large learning rate may prevent stable convergence or cause the model to move away from useful parameter regions. The appropriate value can depend on the architecture, normalization procedure, and properties of the training task.

Four learning rates are evaluated:

$$\eta \in \{0.001, 0.003, 0.01, 0.03\}. \tag{22}$$

For each dataset, model variant, and learning rate, five graph seeds are combined with five initialization seeds. This gives 25 individual training runs per dataset-model-learning-rate condition and 400 runs across the complete comparison. The initialization results are first averaged within each graph seed, followed by a summary across the five graph-level means. Learning-rate selection is performed using validation accuracy rather than test accuracy.

Figure 13 compares the resulting test accuracies. The two datasets exhibit substantially different sensitivity patterns.

For controlled PubMed at $h = 0.1$, eight-layer GraphSAGE remains close to 40% test accuracy throughout the tested learning-rate range. Increasing the learning rate from 0.001 to 0.03 does not produce a meaningful recovery of the baseline in this experiment. This is important because the severe predictive deficit persists across several optimization step sizes rather than appearing only at a single learning rate.

GraphSAGE+PairNorm behaves differently on the same PubMed graphs. Its accuracy improves as the learning rate increases from the smallest evaluated value and reaches approximately the low-to-mid 80% range at the larger learning rates. The resulting separation between baseline GraphSAGE and PairNorm remains large. PairNorm therefore improves the predictive outcome throughout the tested configurations, although the magnitude of the improvement depends on the learning rate.

The checkpoint-selection behavior provides additional context. In several of the low-homophily PubMed baseline runs, the best validation checkpoint occurs at the first few training epochs, whereas the PairNorm models select substantially later checkpoints. This difference indicates that the two variants follow dif-

ferent training trajectories. It is consistent with PairNorm changing the optimization process as well as the representation geometry.

Nevertheless, the early selected baseline checkpoints do not by themselves establish whether optimization difficulty or oversmoothing is the dominant mechanism. The persistent near-degenerate representation measurements in the associated controlled experiments support a representation-collapse interpretation, while the checkpoint dynamics indicate that optimization and representation geometry cannot be treated as independent explanations without further intervention.

Roman-Empire presents a less stable pattern. Baseline GraphSAGE improves as the learning rate increases from 0.001 toward 0.01, but its performance falls again at 0.03. The PairNorm variant follows a different trend and achieves its strongest performance toward the higher end of the tested range. Thus, the relative ranking of the two models depends more strongly on the learning-rate choice than in PubMed.

This contrast matters for interpreting depth-related performance degradation. A model that performs poorly under one learning rate but substantially better under another cannot be described as exhibiting an optimization-independent failure. In Roman-Empire, the tested optimization settings provide an important competing explanation for part of the observed performance variation.

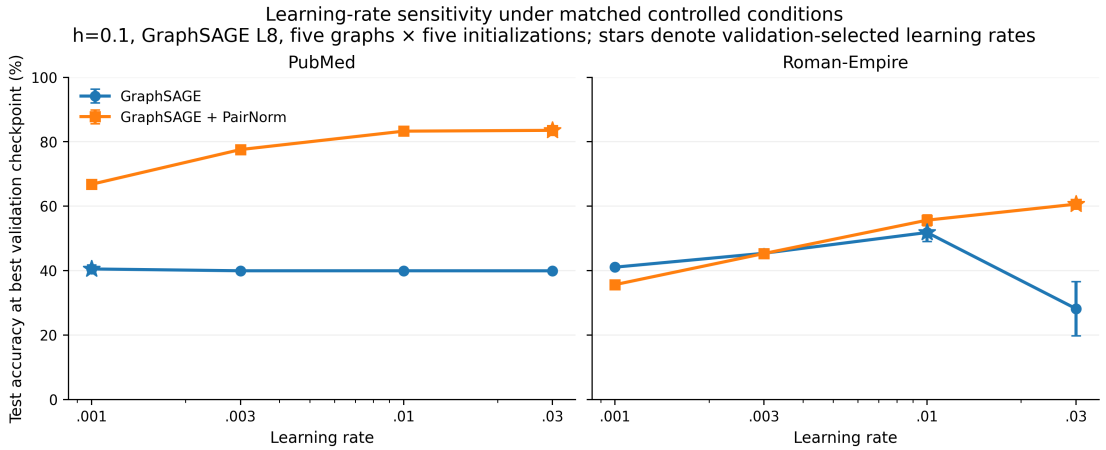


Figure 13: Learning-rate sensitivity of eight-layer GraphSAGE with and without PairNorm on controlled PubMed and Roman-Empire graphs at $h = 0.1$. Results are summarized across five graph seeds after averaging five initialization seeds within each graph. Star markers identify learning rates selected exclusively by validation accuracy.

The experiment therefore distinguishes two kinds of failure within the tested learning-rate grid. PubMed exhibits a robust baseline deficit that is not recovered by changing the learning rate within the evaluated range. Roman-Empire exhibits considerably stronger optimization sensitivity, so conclusions based on a single learning rate would be less reliable.

These observations do not imply that PubMed GraphSAGE would fail under every possible optimizer, learning-rate schedule, initialization, or training procedure. The conclusion is restricted to the tested settings. However, within that range, the persistence of both predictive degradation and representation compression provides stronger evidence for an oversmoothing-related regime than would an isolated accuracy drop.

The next experiment examines PairNorm itself more closely. In particular, it investigates whether the two operations introduced in Section 2.5 have comparable effects or whether different components explain

the successful interventions.

9 PairNorm Component Ablation

PairNorm combines two operations: centering the node representations and rescaling their global spread. The original method applies these operations together [ZA20]. The previous experiments showed that the complete normalization can produce large performance improvements in some conditions while providing little benefit in others.

To investigate whether centering and scaling contribute equally, the component-ablation study compares four configurations: an unnormalized GraphSAGE baseline, centering alone, scaling alone, and full PairNorm. The center-only and scale-only configurations are diagnostic variants introduced for this thesis; they are not presented as separate methods proposed in the original PairNorm publication.

The ablation is evaluated in three contrasting conditions. The first is controlled PubMed at $h = 0.1$, where the deep baseline exhibits severe predictive and representation collapse. The second is controlled Roman-Empire at $h = 0.9$, where the baseline already performs very well and PairNorm changes representation geometry without improving accuracy. The third is the synthetic cSBM at $h = 0.1$ with feature signal 0.50, where the deep baseline can collapse while PairNorm achieves nearly perfect classification.

All three conditions use eight-layer GraphSAGE and a hidden dimension of 128. PubMed and Roman-Empire use learning rate 0.01, while the selected cSBM ablation uses learning rate 0.03. These values define the particular ablation conditions and should not be confused with the default learning rate used in the primary cSBM factorial.

Each condition contains five graph seeds and five initialization seeds. Figure 14 first averages initialization results within each graph seed and then summarizes the five graph-level means. Table 8 instead reports the mean and standard deviation across the 25 individual runs. The two presentations describe the same experiment records but use different aggregation levels.

9.1 Ablation Results

The controlled PubMed condition shows that the scaling operation is particularly important in the observed successful intervention.

The unnormalized baseline reaches 39.94% test accuracy. Applying centering alone does not improve this result: the center-only variant also reaches 39.94%. In contrast, scaling alone raises accuracy to $80.82 \pm 1.00\%$, while full PairNorm reaches $83.28 \pm 0.52\%$. Thus, most of the predictive recovery in this condition is already present when the scale operation is used without the complete normalization.

The representation diagnostics show a corresponding difference. Baseline GraphSAGE has NDE near zero, and centering alone produces an even smaller recorded value. Scaling alone raises NDE to approximately 1.967, while full PairNorm reaches approximately 2.689. Effective rank similarly increases to approximately 3.719 for scale-only and 4.539 for full PairNorm.

These findings indicate that merely subtracting the global mean does not prevent the severe compression observed in the PubMed condition. The scale operation, in contrast, is associated with both greater representation diversity and a substantial accuracy improvement. The additional difference between scale-only and full PairNorm suggests that centering can still affect the final result when used together with scaling.

A numerical qualification is necessary for the center-only effective-rank result. The stored value is reported as zero because of the diagnostic's handling of an almost degenerate representation. Under the mathematical effective-rank definition for a nonzero matrix, the value cannot be below one. The reported zero should therefore be understood as a numerical degeneracy indicator rather than a literal effective rank satisfying the nonzero-matrix definition.

Roman-Empire at $h = 0.9$ behaves differently. The baseline reaches $98.38 \pm 0.29\%$ accuracy, and all three normalization variants remain close to this value. Center-only reaches $98.23 \pm 0.37\%$, scale-only reaches $97.86 \pm 0.35\%$, and full PairNorm reaches $98.21 \pm 0.22\%$. None of these operations produces a useful improvement over the baseline in the tested condition.

The representation geometry nevertheless changes substantially. Effective rank decreases from approximately 9.895 to 1.894 under centering alone, while scaling alone increases it to 14.796 and full PairNorm to 16.413. NDE also changes across the variants, increasing from 0.073 for the baseline to 0.173 under full PairNorm. Thus, the normalization components can alter the measured geometry without improving classification accuracy.

This result reinforces the earlier Roman-Empire counterexample. A model that already separates the relevant classes effectively does not necessarily benefit from additional normalization, even when that normalization substantially changes the global structure of its hidden representations.

The synthetic cSBM condition provides a third, distinct pattern. The unnormalized baseline reaches $54.00 \pm 13.73\%$, reflecting a low mean accuracy and considerable variability. Centering alone raises accuracy to $99.87 \pm 0.20\%$. Scaling alone reaches $99.68 \pm 0.42\%$, while full PairNorm achieves $99.87 \pm 0.24\%$.

Unlike controlled PubMed, where centering alone fails, the synthetic cSBM task is almost perfectly recovered by either component individually. The result therefore does not support a universal explanation in which only scaling is responsible for successful normalization.

The cSBM representation metrics also demonstrate that effective rank is not a sufficient proxy for predictive success. The baseline effective rank is approximately 2.389, while the center-only and scale-only variants have lower mean effective ranks of approximately 1.760 and 1.301, despite achieving almost perfect accuracy. Full PairNorm reaches an effective rank of approximately 1.910. Therefore, a model can achieve excellent classification performance without maximizing the measured effective dimensionality of its hidden representations.

In contrast, normalized Dirichlet energy increases substantially for all successful cSBM variants. However, this does not establish that the energy increase causes their predictive improvement. Both measurements respond to the intervention, and the normalization operations can change several properties of the model simultaneously.

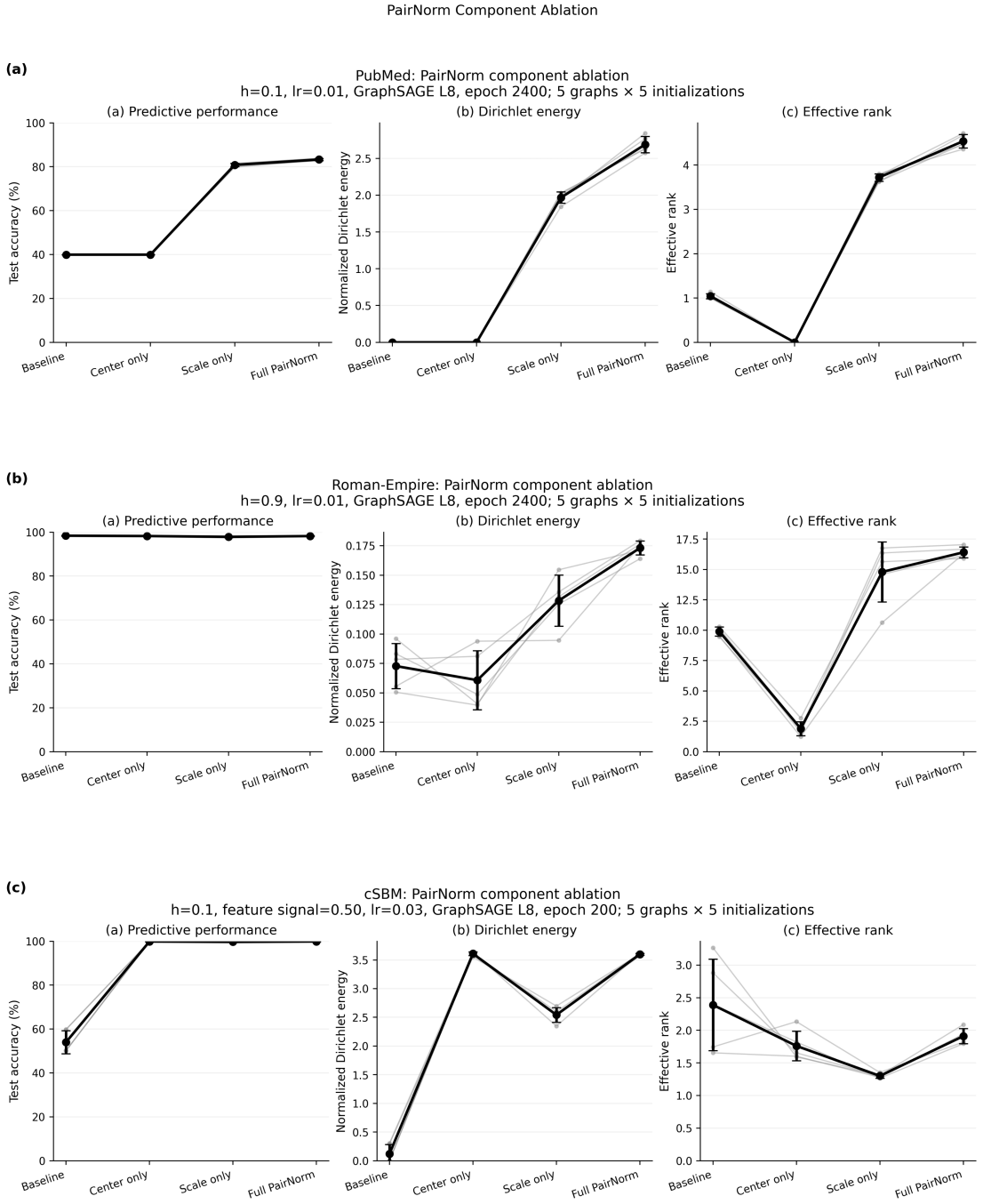


Figure 14: PairNorm component ablations for controlled PubMed, Roman-Empire, and cSBM conditions. Each row compares predictive accuracy, normalized Dirichlet energy, and effective rank across baseline, center-only, scale-only, and full PairNorm variants. Results are first averaged over five initializations within each graph seed, with graph-level variability shown by the error bars.

Table 8: PairNorm component-ablation results. Values are mean ± standard deviation across 25 individual runs per condition and variant. Accuracy is reported at the validation-selected checkpoint, while representation diagnostics are measured at the fixed final epoch.

Condition	Variant	LR	Test accuracy	NDE	Effective rank
PubMed ($h = 0.1$)	Baseline	0.01	$39.94 \pm 0.00\%$	$(5.34 \pm 26.1) \times 10^{-16}$	1.043 ± 0.145
	Center only	0.01	$39.94 \pm 0.00\%$	$(6.17 \pm 28.3) \times 10^{-31}$	0.000 ± 0.000
	Scale only	0.01	$80.82 \pm 1.00\%$	1.967 ± 0.121	3.719 ± 0.214
	Full PairNorm	0.01	$83.28 \pm 0.52\%$	2.689 ± 0.149	4.539 ± 0.211
Roman-Empire ($h = 0.9$)	Baseline	0.01	$98.38 \pm 0.29\%$	0.073 ± 0.028	9.895 ± 0.763
	Center only	0.01	$98.23 \pm 0.37\%$	0.061 ± 0.043	1.894 ± 0.927
	Scale only	0.01	$97.86 \pm 0.35\%$	0.128 ± 0.030	14.796 ± 3.171
	Full PairNorm	0.01	$98.21 \pm 0.22\%$	0.173 ± 0.017	16.413 ± 1.117
cSBM ($h = 0.1$, FS=0.50)	Baseline	0.03	$54.00 \pm 13.73\%$	0.120 ± 0.415	2.389 ± 2.111
	Center only	0.03	$99.87 \pm 0.20\%$	3.610 ± 0.049	1.760 ± 0.351
	Scale only	0.03	$99.68 \pm 0.42\%$	2.542 ± 0.203	1.301 ± 0.061
	Full PairNorm	0.03	$99.87 \pm 0.24\%$	3.600 ± 0.038	1.910 ± 0.308

Note. LR = learning rate; FS = feature signal; NDE = normalized Dirichlet energy. The zero effective-rank value for the degenerate PubMed center-only representation reflects the numerical convention of the stored diagnostic. Figure 14 uses graph-level aggregation, whereas this table summarizes individual runs.

9.2 Interpretation of the Components

The three conditions reveal that the contribution of PairNorm's centering and scaling operations is domain-dependent.

In controlled PubMed at low homophily, scaling is the main successful component. Centering alone does not recover the predictive performance of the collapsed baseline. In the synthetic cSBM task, either centering or scaling alone is sufficient to obtain near-perfect accuracy in the tested condition. In high-homophily Roman-Empire, neither component produces a meaningful predictive advantage because the baseline already performs very well.

These differences are incompatible with the idea that one fixed normalization operation is universally responsible for preventing deep-GNN failure. They also show that preserving a particular geometric statistic is not equivalent to preserving all task-relevant information.

The component ablation strengthens the central conclusion of the preceding experiments. PairNorm can be highly effective under specific conditions, but its predictive effect depends on the relationship between normalization, model depth, graph structure, available information, and optimization. The successful interventions should therefore be interpreted as condition-specific evidence rather than as a general proof that representation spread alone determines GNN performance.

10 Oversmoothing Synthesis

The empirical oversmoothing analysis was designed to answer RQ2 directly: when is depth-related predictive degradation accompanied by representation collapse, and under which conditions does PairNorm mitigate both?

The results support a conditional explanation rather than a universal one.

The broad real-world screening showed that increasing the depth range can substantially reduce predictive accuracy on some datasets, while other datasets retain competitive performance. The size of this degradation varies across graph-learning tasks, model families, and selected configurations. In particular, Amazon-Computers, Roman-Empire, and Actor exhibit pronounced decreases in the deeper screening phase, whereas Coauthor-Physics and PubMed are comparatively stable. These observations provide evidence for heterogeneous depth sensitivity, but they do not establish oversmoothing as its universal cause.

The matched selected-four experiments demonstrated that PairNorm can reduce or increase depth-related degradation. Amazon-Ratings and Squirrel show positive depth interactions under PairNorm, while Roman-Empire and Actor show negative interactions. The representation diagnostics further demonstrate that increased NDE or effective rank does not guarantee stronger prediction. Actor is especially informative because measured representation diversity increases while depth-related performance becomes worse.

The controlled PubMed experiments provide stronger evidence for an oversmoothing-related mechanism. At low homophily, deep GraphSAGE exhibits severe predictive degradation together with extremely small normalized Dirichlet energy and effective rank close to one. PairNorm preserves greater representation diversity and substantially improves test accuracy. The temporal measurements indicate that the compressed baseline representation regime persists throughout much of training rather than appearing only as an isolated final-epoch measurement.

The synthetic cSBM experiments reproduce a particularly clear version of this phenomenon. Under strong heterophily and the tested feature signals, GraphSAGE performs very well at seven layers but collapses

to approximately chance-level accuracy at eight layers and remains degraded in the deeper follow-up probes. PairNorm prevents the predictive failure and maintains substantially greater pairwise representation separation. This provides one of the strongest controlled associations between depth, representation collapse, and predictive degradation observed in this thesis.

However, the counterexamples are equally important for the interpretation. In high-homophily Roman-Empire, PairNorm changes representation geometry without improving an already successful classifier. In the synthetic cSBM, preserving representation diversity does not necessarily recover prediction when the tested graph-feature condition provides insufficient exploitable class information. The component ablations also show that near-perfect accuracy can occur with comparatively low effective rank, so no single representation statistic is sufficient to determine predictive usefulness.

The learning-rate sensitivity experiment provides an additional qualification. Controlled PubMed GraphSAGE remains severely degraded across the tested learning-rate grid, while Roman-Empire exhibits substantial sensitivity to the optimization setting. Thus, an apparent depth failure may be relatively robust to the tested optimization changes in one condition and much more optimization-dependent in another. This distinction argues against inferring a representation-level cause from test accuracy alone.

These findings are consistent with the broader caution in the literature that mitigating a measured oversmoothing statistic need not guarantee stronger deep-GNN performance [RBM23, KDL26, ARE26]. The experiments in this thesis add controlled comparisons showing both where geometric and predictive recovery coincide and where they separate.

RQ2 can therefore be answered as follows. Oversmoothing is a useful explanation for deep-GNN failure in specific regimes where increasing depth is accompanied by persistent representation compression and a substantial predictive decline, especially when a matched normalization intervention prevents both. Controlled low-homophily PubMed and the strongly heterophilous deep cSBM configurations provide the clearest supporting evidence. Nevertheless, representation diversity is not sufficient for good classification, and PairNorm is not universally beneficial. Its effect depends on the graph-learning task, topology, feature information, optimization regime, and implementation.

The evidence therefore supports treating oversmoothing as one possible mechanism of depth-related degradation rather than its general explanation. This conclusion also informs RQ4, which requires oversmoothing to be assessed alongside alternative explanations, including optimization, graph structure, and the availability of task-relevant information.

The next part of the thesis extends this mechanism-oriented approach to long-range information propagation. Instead of focusing primarily on the contraction of node representations, it examines whether information originating from distant nodes is effectively transmitted through the graph and how structural bottlenecks influence that process.

References

[ARE26] Adrian Arnaiz-Rodriguez and Federico Errica. Oversmoothing, "Oversquashing", Heterophily, Long-Range, and More: Demystifying Common Beliefs in Graph Machine Learning. In *International Conference on Learning Representations*, 2026.

[DGGB+23] Francesco Di Giovanni, Lorenzo Giusti, Federico Barbero, Giulia Luise, Pietro Lio, and Michael M. Bronstein. On Over-Squashing in Message Passing Neural Networks: The Impact of Width, Depth, and Topology. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 7865–7885. PMLR, 2023.

[DSMM18] Yash Deshpande, Subhabrata Sen, Andrea Montanari, and Elchanan Mossel. Contextual Stochastic Block Models. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[GSR+17] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural Message Passing for Quantum Chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1263–1272. PMLR, 2017.

[Hol79] Sture Holm. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[HYL17] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems*, volume 30, pages 1024–1034, 2017.

[KDL26] Niklas Kormann, Benjamin Doerr, and Johannes F. Lutzeyer. Position: Don't be Afraid of Over-Smoothing And Over-Squashing. *arXiv preprint arXiv:2601.07419*, 2026.

[KW17] Thomas N. Kipf and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*, 2017.

[Lak13] Daniël Lakens. Calculating and Reporting Effect Sizes to Facilitate Cumulative Science: A Practical Primer for t-tests and ANOVAs. *Frontiers in Psychology*, 4:863, 2013.

[LHW18] Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper Insights Into Graph Convolutional Networks for Semi-Supervised Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pages 3538–3545, 2018.

[Lou20] Andreas Loukas. What Graph Neural Networks Cannot Learn: Depth vs Width. In *International Conference on Learning Representations*, 2020.

[MS02] Sergei Maslov and Kim Sneppen. Specificity and Stability in Topology of Protein Networks. *Science*, 296(5569):910–913, 2002.

[OS20] Kenta Oono and Taiji Suzuki. Graph Neural Networks Exponentially Lose Expressive Power for Node Classification. In *International Conference on Learning Representations*, 2020.

[PKD+23] Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. A Critical Look at the Evaluation of GNNs Under Heterophily: Are We Really Making Progress? In *International Conference on Learning Representations*, 2023.

[PWC+20] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. Geom-GCN: Geometric Graph Convolutional Networks. In *International Conference on Learning Representations*, 2020.

[RAS21] Benedek Rozemberczki, Carl Allen, and Rik Sarkar. Multi-Scale Attributed Node Embedding. *Journal of Complex Networks*, 9(2):cnab014, 2021.

[RBM23] T. Konstantin Rusch, Michael M. Bronstein, and Siddhartha Mishra. A Survey on Oversmoothing in Graph Neural Networks. Technical Report 2023-17, Seminar for Applied Mathematics, ETH Zurich, 2023.

[RV07] Olivier Roy and Martin Vetterli. The Effective Rank: A Measure of Effective Dimensionality. In *2007 15th European Signal Processing Conference*, pages 606–610, 2007.

[SMBG18] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. Pitfalls of Graph Neural Network Evaluation. *Relational Representation Learning Workshop, NeurIPS 2018*, 2018.

[VCC+18] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph Attention Networks. In *International Conference on Learning Representations*, 2018.

[XLT+18] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. Representation Learning on Graphs with Jumping Knowledge Networks. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 5453–5462. PMLR, 2018.

[YCS16] Zhilin Yang, William W. Cohen, and Ruslan Salakhutdinov. Revisiting Semi-Supervised Learning with Graph Embeddings. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 40–48. PMLR, 2016.

[ZA20] Lingxiao Zhao and Leman Akoglu. PairNorm: Tackling Oversmoothing in GNNs. In *International Conference on Learning Representations*, 2020.

[ZDHT26] Kaicheng Zhang, Piero Deidda, Desmond J. Higham, and Francesco Tudisco. Are We Measuring Oversmoothing in Graph Neural Networks Correctly? In *International Conference on Learning Representations*, 2026.

[ZYZ+20] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. Beyond Homophily in Graph Neural Networks: Current Limitations and Effective Designs. In *Advances in Neural Information Processing Systems*, volume 33, 2020.